This is a copy of a shared ChatGPT chat. It won't be added to memory.

**Report conversation**

Monday 10:12 AM

**You said:**

i run an analysis of a model as: VT_model <- glmmTMB(Vt_STPD ~ Side + BM + Sp+ (1 | ID), data = VT_all, family = Gamma(link = "log")) where Vt_STPD is tidal volume, side is which side the animal is laying on, BM is body mass and Sp the species. ID is individual animal and this model fails in the KS test when i run the simulateResiduals. The plot shows a curvature against the line of identity

**ChatGPT said:**

A curved DHARMa QQ plot indicates that the model does not reproduce the distribution of the response correctly. In your case, the most likely issue is that the relationship between tidal volume and body mass is not linear on the response scale.

Because tidal volume commonly scales allometrically with body mass, try modelling body mass on a logarithmic scale:

R

```r
VT_all$logBM <- log(VT_all$BM)

VT_model2 <- glmmTMB(
  Vt_STPD ~ Side + logBM + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

You should also consider that the body-mass relationship may differ among species:

R

```r
VT_model3 <- glmmTMB(
  Vt_STPD ~ Side + logBM * Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

The `Side × Sp` interaction may also be biologically relevant if the effect of lying side differs among species:

R

```r
VT_model4 <- glmmTMB(
  Vt_STPD ~ Side * Sp + logBM * Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

Then compare the models using AIC and residual diagnostics:

R

```r
library(DHARMa)

res <- simulateResiduals(VT_model2, n = 1000)
plot(res)

testUniformity(res)
testDispersion(res)
testOutliers(res)
```

You can specifically inspect the body-mass relationship:

R

```r
plotResiduals(res, VT_all$BM)
plotResiduals(res, VT_all$logBM)
```

A useful additional check is whether the relationship is nonlinear even after log-transforming body mass:

R

```r
VT_model_spline <- glmmTMB(
  Vt_STPD ~ Side + splines::ns(logBM, df = 2) + Sp + (1 | ID),
  data = VT_all,
```

```
  family = Gamma(link = "log")
)
```

Alternatively, use a quadratic term:

```R
VT_model_quad <- glmmTMB(
  Vt_STPD ~ Side + logBM + I(logBM^2) + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

Other possibilities to check are:

- the variance increases more than expected with the mean;
- repeated measurements from the same individual require a random slope, for example `(Side | ID)`;
- there are species-specific differences in variance;
- the Gamma distribution is inappropriate, in which case a lognormal model may fit better.

For example:

```R
VT_model_lognormal <- glmmTMB(
  Vt_STPD ~ Side + logBM + Sp + (1 | ID),
  data = VT_all,
  family = gaussian(link = "log")
)
```

The first model I would try is therefore:

```R
VT_model2 <- glmmTMB(
  Vt_STPD ~ Side + log(BM) + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

A curvature in the DHARMa QQ plot is most often a sign of a missing nonlinear effect, and `BM` is the strongest candidate here.

Worked for 22s

**You said:**

but should i not use another link function if I am going to use log-transforms and should i use the lme function instead

**ChatGPT said:**

No. Transforming the predictor `BM` does not require changing the link function.

These are separate decisions:

- `log(BM)` describes the relationship between body mass and tidal volume.
- `link = "log"` describes how the expected tidal volume is related to the linear predictor.

A sensible model is therefore:

```r
VT_model2 <- glmmTMB(
  Vt_STPD ~ Side + log(BM) + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

This models tidal volume as positive and allows an approximately allometric relationship with body mass:

$$V_T \propto BM^{\beta}$$

You would generally not switch to an identity link simply because `BM` was log-transformed.

`lme()` is appropriate if you log-transform the response and assume normally distributed residuals:

```r
library(nlme)

VT_model_lme <- lme(
  log(Vt_STPD) ~ Side + log(BM) + Sp,
  random = ~1 | ID,
```

```
  data = VT_all,
  method = "REML"
)
```

This is effectively a lognormal mixed-effects model. It can be useful if:

- `log(Vt_STPD)` is approximately normally distributed;
- residual variance is reasonably constant on the log scale;
- the lognormal model gives better DHARMa-equivalent or residual diagnostics.

However, `lme()` does not fit a Gamma model. I would first compare:

R

```
VT_gamma <- glmmTMB(
  Vt_STPD ~ Side + log(BM) + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)

VT_lognormal <- glmmTMB(
  Vt_STPD ~ Side + log(BM) + Sp + (1 | ID),
  data = VT_all,
  family = gaussian(link = "log")
)
```

Then inspect residuals and fitted-versus-observed plots for both models. The choice should be based on which model better represents the data, not on the fact that body mass was log-transformed.

One further issue is that the effect of `Side` may vary between individuals. If each animal was measured on both sides, consider:

R

```
VT_gamma_rs <- glmmTMB(
  Vt_STPD ~ Side + log(BM) + Sp + (1 + Side | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

provided there are enough repeated measurements per individual and the model converges.

Worked for 19s

**You said:**

it made it worse

**ChatGPT said:**

That is useful information: it suggests that the body-mass relationship is not simply improved by using `log(BM)`. You do not need to switch to `lme()` yet.

Your original model assumes:

$$E(V_T) = \exp(a + b\,BM)$$

whereas the log-body-mass model assumes:

$$E(V_T) = \exp(a)\,BM^b$$

If the second model performs worse, the data may not follow a simple allometric relationship, or the problem may be elsewhere.

First identify where the residual curvature occurs:

```R
res <- simulateResiduals(VT_model, n = 1000)

plotResiduals(res, VT_all$BM)
plotResiduals(res, VT_all$Sp)
plotResiduals(res, VT_all$Side)

testDispersion(res)
testOutliers(res)
```

If the curvature is against `BM`, try a flexible nonlinear effect:

```R
VT_spline <- glmmTMB(
  Vt_STPD ~ Side + splines::ns(BM, df = 2) + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

or:

R

```r
VT_spline2 <- glmmTMB(
  Vt_STPD ~ Side + splines::ns(BM, df = 3) + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

If the body-mass relationship differs among species, this may be more appropriate:

R

```r
VT_species_slope <- glmmTMB(
  Vt_STPD ~ Side + BM * Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)
```

Also check whether the problem is unequal dispersion between species:

R

```r
VT_disp <- glmmTMB(
  Vt_STPD ~ Side + BM + Sp + (1 | ID),
  dispformula = ~ Sp,
  data = VT_all,
  family = Gamma(link = "log")
)
```

Then compare the models using AIC and residual plots:

R

```r
AIC(VT_model, VT_spline, VT_species_slope, VT_disp)
```

Do not choose `lme()` merely because the Gamma model fails the KS test. `lme()` would be appropriate only if `log(Vt_STPD)` produces approximately normal, homoscedastic residuals:

R

```r
VT_lme <- nlme::lme(
  log(Vt_STPD) ~ Side + BM + Sp,
  random = ~ 1 | ID,
```

```
  data = VT_all
)
```

The most important next step is to see whether the DHARMa curvature is associated with `BM`, `Sp`, `Side`, fitted values, or dispersion. The QQ plot alone cannot identify which part of the model is wrong.

Worked for 20s

---

**You said:**

what about another link function?

**ChatGPT said:**

Yes, you can try another link function. For a Gamma model, the usual choices are:

R

```r
# Log link
m_log <- glmmTMB(
  Vt_STPD ~ Side + BM + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "log")
)

# Identity link
m_identity <- glmmTMB(
  Vt_STPD ~ Side + BM + Sp + (1 | ID),
  data = VT_all,
  family = Gamma(link = "identity")
)

# Inverse link
m_inverse <- glmmTMB(
  Vt_STPD ~ Side + BM + Sp + (1 | ID),
  data = VT_all,
```

The links imply different relationships:

- `log`: multiplicative effects; usually the safest default for positive biological measurements.
- `identity`: tidal volume changes linearly with the predictors.
- `inverse`: the reciprocal of tidal volume changes linearly; usually less biologically intuitive.

Check convergence and residuals for each:

R

```r
models <- list(
  log = m_log,
  identity = m_identity,
  inverse = m_inverse
)

lapply(models, function(m) {
  r <- simulateResiduals(m, n = 1000)
  c(
    uniformity = testUniformity(r)$p.value,
    dispersion = testDispersion(r)$p.value
  )
})
```

Also compare:

R

```r
AIC(m_log, m_identity, m_inverse)
```

I would not automatically choose the model with the largest KS-test p-value. First ensure that it converges, has no extreme fitted values, and shows no systematic residual pattern. The Gamma log-link model is generally the most defensible starting point because tidal volume is positive and often changes proportionally with body size.

If all three Gamma links fail, the problem may be the distribution rather than the link. Then compare with a lognormal model:

R

```r
m_lognormal <- glmmTMB(
  Vt_STPD ~ Side + BM + Sp + (1 | ID),
  data = VT_all,
  family = gaussian(link = "log")
)
```

A useful next comparison would therefore be:

R

```r
AIC(m_log, m_identity, m_inverse, m_lognormal)
```

If you show me the DHARMa residual plots for the different models, especially residuals versus fitted values and `BM`, I can help identify whether the problem is the link, the body-mass relationship, or the variance structure.

Worked for 17s

---

Monday 1:05 PM

**You said:**

what is the best way to work on some matlab code that is part of a project, we do this in "chat" or is there some other place that is better?

**ChatGPT said:**

For an ongoing MATLAB project, this chat is a good place to work, provided the project files are available in the workspace. The best setup is:

1. Keep the actual project in a folder or Git repository.
2. Include the `.m` files, scripts, functions, and—where practical—small example data.
3. Ask me first to inspect and explain the project structure without changing anything.
4. Then work on one defined task at a time: debugging, refactoring, adding an analysis, improving plots, or documenting code.
5. Keep MATLAB or VS Code as the main editor and use this chat for reasoning, code changes, and review.

A good first message would be:

> Please inspect this MATLAB project and explain the folder structure, identify the main entry point, and describe how the analysis flows. Do not modify any files yet.

After that, you could ask:

> Please modify the code so that [specific change]. Preserve the existing outputs and explain which files you changed.

For a larger project, a dedicated ChatGPT Project or a separate conversation just for that codebase is preferable, since it keeps the context focused. A single chat is usually best for an individual analysis or debugging task.

If you upload or place the MATLAB project in the workspace, I can inspect the files and work directly on them. I can reliably edit and review `.m` files; execution will depend on whether MATLAB or a compatible environment is available here.

Worked for 16s

**You said:**

the code is old and would need some revision but i also want to change part of the structure. It has been published several time, updates during the last 18 years so it is really patchy with changes and revisions but i now specifically would like to make a modification. I do not know how github works so not sure that is the best, but if you tell me that is the best structure, i will read up on it

**ChatGPT said:**

For a codebase that has evolved over 18 years, Git is probably the best structure. It will let you preserve the published versions, make the new modification safely, and see exactly what changed.

GitHub is simply an online place to store a Git repository. You can make the repository private, so the code does not need to be public. You do not need to learn the command line; GitHub Desktop provides a graphical interface.

I would recommend this structure:

Plain text

```
project-name/
├── README.md
├── project-name.prj
├── src/              % Current MATLAB functions
├── scripts/          % Analysis and plotting scripts
├── legacy/           % Old versions retained for reference
├── tests/            % Test scripts
├── example_data/     % Small data set for testing
├── figures/
└── data/             % Usually excluded from GitHub
```

The important principle is not to overwrite the old published code. Instead:

- preserve the original versions;
- create a new cleaned-up working version;
- record each major published version as a Git “tag”;
- make the requested modification in a separate branch;

- keep raw data and large output files outside the repository or exclude them with `.gitignore`.

A practical workflow would be:

1. Make a complete backup of the existing code.
2. Put the current code into a private GitHub repository.
3. Label the existing state as something like `legacy-original`.
4. Organize the files without changing their behaviour.
5. Create a new version for the structural modification.
6. Test the new and old versions against a small example dataset.
7. Keep a short change log explaining what was altered and why.

You do not need to understand all of Git before starting. I can guide you through setting it up and help decide how the old files should be organized. For this project, I would use GitHub as the durable home for the code and this chat as the place where we inspect, revise, and document it.

Worked for 15s

---

**You said:**

So now I have a GitHub account and I have located the 60 files that make up the code. Can you help me set this up, with a legacy folder as you propose and then we can start to work on this? can we date the legacy folder, or how does that work?

**ChatGPT said:**

Yes. I can help you set this up step by step.

The date should primarily be recorded using Git, rather than only in the folder name:

- Git commit: records exactly when the files were imported.
- Git tag: gives that state a permanent name, such as `legacy-2026-10-12`.
- Folder name: can provide additional clarity, such as `legacy/2026-10-12_original`.

I suggest this initial structure:

Plain text

```
project-name/
├── legacy/
```

```
│   └── 2026-10-12_original/
│       └── [the 60 untouched files]
├── src/
├── scripts/
├── tests/
├── README.md
└── .gitignore
```

However, we should first preserve the 60 files exactly as they are, without reorganizing or renaming them. Old MATLAB code often depends on file locations, names, or relative paths.

The setup process would be:

1. Create a private GitHub repository.
2. Upload or copy the 60 files into the repository.
3. Place an untouched copy in `legacy/2026-10-12_original/`.
4. Make the first commit, for example:

   `Import original MATLAB code recovered on 2026-10-12`
5. Create a tag named:

   `legacy-2026-10-12`
6. Create a working copy or branch for the revisions and structural changes.

The `legacy` folder should then be treated as an archival snapshot. We can work on the new version without risking the original code. Git will also allow us to return to the exact original state at any time.

The easiest method for you is probably GitHub Desktop rather than the command line. If you upload the 60 files here as a ZIP file, I can first inspect them, identify the likely entry points and dependencies, and propose the exact folder structure before you commit anything.

Worked for 21s

---

**You said:**

i created a repositiry and named it Gas-dynamics-model and if i drop the matlab m-files into the main folder, how do i setup the directory legacy/2026-10-12-original/?

Worked for 36s >

---

**ChatGPT said:**

Yes. Since the files are already in the GitHub repository, use GitHub Desktop to make the change locally. Cloning creates a local copy that stays synchronized with the GitHub repository. GitHub Docs

## 1. Clone the repository

On the GitHub page for `Gas-dynamics-model`:

1. Click **Code**.
2. Choose **Open with GitHub Desktop**.
3. Select where to save the local copy.
4. Click **Clone**.

Alternatively, in GitHub Desktop use **File → Clone repository**. GitHub Docs

## 2. Create the legacy folder

In GitHub Desktop, choose:

**Repository → Show in Explorer**

Inside the repository folder, create:

```
Plain text

legacy
└── 2026-10-12-original
```

Then move the MATLAB files from the main folder into:

```
Plain text

Gas-dynamics-model/legacy/2026-10-12-original/
```

Leave files such as `README.md` in the main folder.

Do not rename or edit the MATLAB files at this stage.

## 3. Commit the change

Return to GitHub Desktop. It should show the files as moved rather than simply deleted and recreated.

Use this commit message:

```
Archive original MATLAB code as of 2026-10-12
```

Click **Commit to main**, followed by **Push origin**. Committing records the change locally, and pushing sends it to GitHub. GitHub Docs

### 4. Create a permanent label

The folder date helps you recognize the archive, but Git also preserves the exact historical state. Later, we can create a tag such as:

```
legacy-2026-10-12
```

That gives this version a permanent label, even if the folder structure changes later. GitHub Desktop supports creating and pushing tags associated with commits. GitHub Docs

Once this is done, tell me whether the files are now visible in the dated folder, and we can inspect the project structure before making any revisions.

**You said:**

where do i create the permanent label?

Worked for 13s >

**ChatGPT said:**

You create the permanent label in GitHub Desktop, after committing and pushing the folder move.

1. Open **GitHub Desktop** and select `Gas-dynamics-model`.
2. Click **History** in the left sidebar.
3. Find the commit:

   `Archive original MATLAB code as of 2026-10-12`
4. Right-click that commit.
5. Choose **Create Tag…**
6. Enter:

```
legacy-2026-10-12
```

7. Click **Create Tag**.
8. If GitHub Desktop shows an arrow indicating the tag has not been uploaded, right-click the tag or use **Push origin**.

The tag is attached to that exact commit and permanently identifies that version of the project. GitHub Desktop's current instructions specify creating tags by right-clicking a commit in the **History** view. docs.github.com

You should then be able to see the tag on the GitHub website under the repository's **Tags** or **Releases** area.

**You said:**

ok now we are there, whats next and what did we just do?

**ChatGPT said:**

Excellent. We have now created a safe starting point.

What we just did:

- The 60 original MATLAB files were placed in:

```
Plain text

legacy/2026-10-12-original/
```

- We committed that change, creating a permanent record in Git.
- We created the tag:

```
Plain text

legacy-2026-10-12
```

The tag is like a bookmark to the exact original project state. Even if we later change, rename, or reorganize files, we can always return to that version. GitHub tags are attached to specific commits in the project history. docs.github.com

## The next step

Create a separate working branch so that future changes are isolated from the main branch.

In GitHub Desktop:

1. Click **Current Branch** near the top.
2. Choose **New Branch**.
3. Name it:

```
Plain text

development
```

4. Ensure it is based on `main`.
5. Click **Create Branch**.
6. Click **Publish branch**.

After that:

- `main` will represent the stable archived project.
- `legacy-2026-10-12` identifies the original snapshot.
- `development` is where we will revise and restructure the code.

The next actual code step should be an inspection only. We should identify:

- the main script or entry point;
- which files call which other files;
- required input data;
- generated outputs and figures;
- duplicated or obsolete versions;
- functions that depend on files being in the current directory.

Do not reorganize the files further yet. If you upload a ZIP copy of the repository here, I can inspect the 60 MATLAB files and create a project map before we modify anything.

Worked for 19s

---

**You said:**

how do i know it is based on main? In the "current branch" it says development and main is the default

**ChatGPT said:**

If you created `development` while `main` was selected, then it is based on `main`. The fact that `main` is the default branch does not itself prove it, but it is the normal starting point.

You can verify it in GitHub Desktop:

1. Click **Current Branch** → **main**.
2. Open **History** and note the most recent commit.
3. Switch back to **development**.
4. Open **History** again.

If `development` has no new commits yet, its latest commit should be exactly the same as `main`. That means it was created correctly from `main`.

You can also check online: GitHub should show `development` as a branch containing the same files and commit history as `main`.

Do not worry if the branch selector now says `development`; that simply means you are currently working on the development branch. The `main` branch remains unchanged.

Worked for 12s

---

**You said:**

ok now what should we do next and do we do someting in development?

**ChatGPT said:**

Yes. From now on, we do the work in the `development` branch. We leave `main` and the tagged legacy version unchanged.

The next step should be inspection, not restructuring. We need to understand the old code before moving files again.

## What to do now

1. Confirm that GitHub Desktop shows:

```
Plain text

Current branch: development
```

2. Choose **Repository** → **Show in Explorer**.
3. Make a ZIP copy of the repository folder, including:

```
Plain text

legacy/2026-10-12-original/
```

You can omit the hidden `.git` folder and any large raw data files.

4. Upload that ZIP file here.

I can then examine the 60 MATLAB files and identify:

- the main analysis script;
- functions called by each script;
- required input files;
- output files and figures;
- duplicate or obsolete versions;
- assumptions about the current working directory;
- the safest new structure for the revised version.

After that, we will probably create a clean working structure within `development`, for example:

Plain text

```
src/
scripts/
tests/
example_data/
README.md
```

The original files will remain untouched in `legacy/2026-10-12-original/`.

We will then make one clear change at a time in `development`, committing after each meaningful step. The overall process will be:

1. map the existing code;
2. make the old code reproducible;
3. restructure it;
4. implement your requested modification;
5. compare the revised results with the original results;
6. eventually create a new version tag.

So, for now, the useful next action is to upload the ZIP of the repository.

Worked for 17s

---

**You said:**

I am a bit confused, so in explorer, i copy the folder legacy/2026-10-12-origina/ here in chatgpt? The working

structure you suggest within development, where/how to i create that?

**ChatGPT said:**

You do not need to create the new working structure yet. Let's first inspect the old code.

There are three separate places:

1. **Your local repository**
   This is the folder on your computer managed by GitHub Desktop. This is where we will eventually create `src`, `scripts`, and `tests`.
2. **GitHub.com**
   This is the online backup and version history.
3. **This chat**
   This is where you can upload a copy so I can inspect and help reorganize it.

## For now: upload the legacy code here

In Windows Explorer:

1. Open the local `Gas-dynamics-model` folder.
2. Open:

```
Plain text

legacy/2026-10-12-original/
```

3. Right-click the folder `2026-10-12-original`.
4. Choose **Send to → Compressed (zipped) folder**.
5. Attach the resulting ZIP file to this chat using the paperclip or upload button.

You can omit large data files. The MATLAB `.m` files are the important part. If there are small configuration files or example data files required to understand the code, include those too.

## Where the new structure will be created

After I inspect the code, we will create the new folders inside your local repository while the `development` branch is selected:

```
Plain text

Gas-dynamics-model/
├── legacy/
│   └── 2026-10-12-original/
```

```
├── src/
├── scripts/
├── tests/
└── README.md
```

You would create these folders in Windows Explorer inside the local `Gas-dynamics-model` folder, or I can prepare the revised structure for you. GitHub Desktop will detect the new folders and files, and you will then commit and push them to the `development` branch.

For now, just upload a ZIP copy of `legacy/2026-10-12-original/`. Do not move or modify the files further.

Worked for 17s

---

**You said:**

**2026-10-12-origi…**
File

in explorer at the moment, i only have the folder "legacy" and the hidden folder "git". I do not see "development" and not sure where that exist. Do i know

**ChatGPT said:**

That is normal. `development` is not a folder—it is a Git branch, or a separate version of the same folder. Git stores the branch information inside the hidden `.git` folder.

Your local structure may therefore look like:

Plain text

```
Gas-dynamics-model/
├── legacy/
│   └── 2026-10-12-original/
└── .git/
```

To work on `development`:

1. Open **GitHub Desktop**.
2. Select the repository `Gas-dynamics-model`.
3. Click **Current Branch** at the top.
4. Select `development`.

The Explorer folder will remain the same. Any changes you make there will now belong to the `development` branch. When you commit and push, they will be sent to the development branch on GitHub.

Do not create a physical folder called `development`.

I also received the ZIP file. Once you confirm that GitHub Desktop shows `development` as the current branch, I can inspect the MATLAB files and map out the project before we change its structure.

Worked for 16s

**You said:**

yes it now shows development. Now will you do the inspection? Should I update the readme file with someting?

Worked for 1m 9s >

**ChatGPT said:**

Yes—I inspected the uploaded legacy copy. You do not need to change the README yet, but we now know enough to add a short initial description.

The legacy folder contains 60 MATLAB files. The main apparent entry point is:

Plain text

```
driver_to_beakedwhale_tissue_kin.m
```

The main processing flow appears to be:

Plain text

```
driver_to_beakedwhale_tissue_kin
        ↓
GET_DIVE_DATA_LOAD_TISSUE_KIN
GET_DIVE_SUMMARY_DATA_METABOLIC_RATES
        ↓
beakedwhale_tissue_kinetics1
        ↓
GET_VARIABLES_ZIP_NEW
GET_GAS_FLUX_1
        ↓
```

```
tissue, blood, lung gas calculations
        ↓
SAVEDATA
```

Important findings:

- The code relies heavily on global variables.
- The driver currently processes whales 11 and 12 only.
- It expects several CSV input files that are not included in the ZIP.
- The CSV files are expected to be in the current MATLAB working directory.
- Results are saved as `.mat` files and tab-delimited output.
- There are several older and newer versions of functions mixed together.
- The code has accumulated changes from approximately 2010 through 2026.
- The primary model is currently contained in one large collection of functions rather than a clearly separated `src` / `scripts` structure.

## README

I suggest adding a preliminary README on the `development` branch, but keeping it modest for now. For example:

Markdown

```
# Gas-dynamics-model

MATLAB code for modelling gas kinetics in diving beaked whales.

## Current status

This repository contains a legacy version of a model that has been developed and
modified over several years. The original files are preserved in:

    legacy/2026-10-12-original/

The current development branch is being used to document, reorganize, and revise the
model.

## Main entry point

The current apparent driver script is:
```

You can add this later using the local repository folder while `development` is selected. I recommend that we first determine exactly which input files you have and identify the modification you want to make. Then we can create the new structure accurately rather than moving files prematurely.

**You said:**

how do i add this readme file in development?

**ChatGPT said:**

Create it in the main `Gas-dynamics-model` folder, not inside `legacy`.

## Using Windows Explorer

1. Confirm GitHub Desktop shows `development` as the current branch.
2. Choose **Repository** → **Show in Explorer**.
3. Open the main folder:

```
Plain text

Gas-dynamics-model
```

4. Right-click an empty area and choose **New** → **Text Document**.
5. Rename it exactly:

```
Plain text

README.md
```

   Make sure it does not become `README.md.txt`. If necessary, enable **View** → **Show** → **File name extensions** in Explorer.
6. Open `README.md` with Notepad.
7. Paste the README text I provided.
8. Save and close the file.

The structure should look like:

```
Plain text

Gas-dynamics-model/
├── README.md
├── legacy/
│   └── 2026-10-12-original/
└── .git/
```

## Commit it to `development`

Return to GitHub Desktop. It should show `README.md` as a changed file.

Use:

```
Plain text

Add initial project documentation
```

as the commit message, then click:

1. **Commit to development**
2. **Push origin**

The README will then be stored on the `development` branch on GitHub. Do not modify `main` or the `legacy-2026-10-12` tag.

Worked for 14s

---

**You said:**

okk, done and now what?

**ChatGPT said:**

The next step is to establish a baseline: confirm that the legacy model can still run before we reorganize or modify it.

The driver currently runs whales 11 and 12, so it expects these four CSV files:

```
Plain text

11-Zica-20240227-233396_surfacings_1Hz-data.csv
11-Zica-20240227-233396_surfacings_1Hz-summary.csv

12-Zica-20240227-240128_surfacings_1Hz-data.csv
12-Zica-20240227-240128_surfacings_1Hz-summary.csv
```

Please locate these files on your computer. If they are not confidential or too large, upload them here, preferably as a ZIP file. If they are sensitive or very large, upload only:

- the first 10–20 rows;
- the column names;
- the approximate file sizes;
- information about where the complete files are stored.

For now, do not reorganize the MATLAB files or create `src` and `scripts`. We first need to know whether the old model runs and what inputs it requires. Once we establish that

baseline, we will create the new working structure on the `development` branch and make the requested modification safely.

Worked for 21s

**You said:**

**11-Zica-2024022…**
File

Here are the files, the shorter one is a summary file that describes data used and how the model changes physiology of the whale. The other file is a data file of dive data and activity, which are used to step through the model and then calculate the output. All data files, whale 1-12 are placed inside the main folder when the model is runnning and then there is a loop that selects them one by one

Worked for 37s >

**ChatGPT said:**

I inspected the uploaded files. They match the MATLAB code very well.

The ZIP contains:

Plain text

```
11-Zica-20240227-233396_surfacings_1Hz-data.csv
11-Zica-20240227-233396_surfacings_1Hz-summary.csv

12-Zica-20240227-240128_surfacings_1Hz-data.csv
12-Zica-20240227-240128_surfacings_1Hz-summary.csv
```

The data files contain:

Plain text

```
Time, depth, ODBA, locomotion cost, sonar
```

The summary files contain 13 columns, including:

Plain text

```
surface interval,
dive duration,
start and end rows,
mean metabolic rate,
maximum depth,
mean depth,
sonar
```

This agrees with the functions:

```matlab
GET_DIVE_DATA_LOAD_TISSUE_KIN
GET_DIVE_SUMMARY_DATA_METABOLIC_RATES
```

The current driver loops over whales 11 and 12, so these files are sufficient for an initial test:

```matlab
for i = 11:12
```

### What to do next

For the first baseline run, place these four CSV files temporarily in the same local folder as the MATLAB files:

```
legacy/2026-10-12-original/
```

This is necessary because the old code looks for the filenames directly in the current MATLAB folder.

Then in MATLAB:

1. Set the current folder to:

   ```
   legacy/2026-10-12-original/
   ```

2. Make sure the `development` branch is selected in GitHub Desktop.
3. Run:

```matlab
driver_to_beakedwhale_tissue_kin
```

Do not commit the CSV files to GitHub yet. They are large, and the repository should eventually contain code separately from data. GitHub Desktop may show them as new files, but leave them uncommitted for now.

The important result of this first run is either:

- the model runs and produces output; or
- MATLAB gives an error.

Please copy the complete MATLAB error message here if it fails. That error will tell us the first compatibility or path problem to fix before restructuring the code.

**You said:**

OK, i plasted the data from whales 11 and 12 and then I will open the drive to beahedl whale tissue kin in matlab and run from the legacy/2026-10-12-original/ site and see wha tthe output it and we can decide what to do next. In the end, what the aim now is maybe it can be to clean up some of the old code, but i also want to create code that specifically accounts for minute and alveolar ventilation when the whale is at the surface. Now they animal just breathes from an endless supply at the surface.

**ChatGPT said:**

That is the right next step. Run:

```matlab
driver_to_beakedwhale_tissue_kin
```

from:

```Plain text
legacy/2026-10-12-original/
```

This gives us an unmodified baseline. Note whether it runs successfully, how long it takes, and which output files it creates. If it fails, copy the complete MATLAB error message.

Your longer-term aim is also clear: retain the existing diving gas-kinetics model, but replace the assumption of an unlimited surface gas supply with an explicit surface-breathing model.

At the surface, we would likely need to represent:

$$\dot{V}_E = f \times V_T$$

where $\dot{V}_E$ is minute ventilation, $f$ is breathing frequency, and $V_T$ is tidal volume, and:

$$\dot{V}_A = f \times (V_T - V_D)$$

where $\dot{V}_A$ is alveolar ventilation and $V_D$ is anatomical or physiological dead space.

The revised model could therefore account for:

- tidal volume;
- breathing frequency;
- minute ventilation;
- dead-space ventilation;
- alveolar ventilation;
- inspired gas composition;
- lung volume and residual gas;
- gas exchange during each breath;
- the number and timing of breaths during the surface interval.

The sensible development sequence will be:

1. Confirm that the current legacy model runs.
2. Document exactly how it currently handles surface intervals.
3. Clean up and separate the code into input, model, and output components.
4. Add a modular surface-ventilation function.
5. Compare the revised model with the original model.
6. Test how different ventilation assumptions affect tissue $O_2$, $CO_2$, and $N_2$ dynamics.

For now, run the original driver and send me either the output or the complete error message.

Worked for 16s

Monday 5:42 PM

**You said:**

the first file took about 2 hours to run and the second about 30 minutes. They finished and provided the expected output. These files are now working fine and this is the backup model? What next. Yes, the idea is ok above, but we do not need a "breathing frequency" but we need input from the data stream when the animal breathes and then we "assign" a tidal volume to that breath. Inspired air is always 21%o2 and 79% N2 and we can assign lung details but that is also defined for the animal in the "GET_VARIABLES_LUNG_COLL.m" file. So how do we proceed?

Worked for 1m 16s >

**ChatGPT said:**

Excellent. We now have a successful baseline. The legacy model is preserved by:

- the untouched files in `legacy/2026-10-12-original/`;
- the `main` branch;
- the tag `legacy-2026-10-12`.

So we can always return to the original code. The generated output files are not automatically part of that backup unless they were committed separately, so keep those outputs in a separate results folder.

I also inspected the surface logic. At present, when the whale is at the surface, `update_lung_gas.m` calls:

```matlab
update_N2_case_A_or_C
update_O2_case_A_or_C
update_CO2_case_A_or_C
```

Those functions reset the lung gas to the fixed values in `ALV_SURFACE_VALUE` at every time step. In effect, the lung is continuously connected to an infinite atmospheric reservoir.

Your proposed modification should instead do the following:

```
Plain text

Dive:
    use the existing pressure-dependent gas-kinetics model

Surface, no breath:
    retain the current lung gas and allow ongoing blood-lung exchange

Surface, breath detected:
    replace an appropriate volume of lung gas with inspired gas
    using the assigned tidal volume
```

The new tidal-volume variable should have a new name, such as:

```
matlab

V_TIDAL
```

We should not reuse `VT` or `VTmax`, because in the existing lung-collapse code those variables refer to tracheal/dead-space volume, not tidal volume.

One important issue is that the uploaded data files do not contain a breath-event column. They contain:

```
Plain text

Time
Depth
ODBA
Locomotion cost
Sonar
```

Therefore, before implementing the new surface model, we need to identify where the breath timing comes from. It could be:

- another data stream;
- a manually identified breath-event file;
- a signal from which breaths can be detected;
- or an additional column that was omitted from the uploaded files.

The cleanest future input would be a separate file aligned by time, for example:

```
Plain text

time_s, breath
26751, 0
```

```
26752, 0
26753, 1
26754, 0
```

where `breath = 1` identifies a breath. We could then assign a tidal volume to each breath, initially as a constant animal-specific value and later potentially as a variable value.

### What we should do next

Before changing the model:

1. Identify or upload the data containing breath timing.
2. Make a working copy of the MATLAB code outside the legacy folder.
3. Build and test a short surface-breathing example.
4. Add that new surface-breathing function to the model.
5. Run a shortened version of the whale data before repeating the full two-hour simulations.

The next question is therefore: where does the information indicating that a breath occurred come from?

**You said:**

the data stream for breaths do not currently exist but will have to be a new input variable. The N2_ARRAY, O2_ARRAY, and CO2_ARRAY contains the data for lung gas, and this works fine when the animal is diving, but we need to do something similar and "restrict" exchange in between breaths and then account for how the gas is changing, similar to what we do during diving. Maybe the "easiest" is the set the animal to "always" diving and then briefly get to the surface and then dive again, so we assume that a specific volume (tidal volume) is taken up during the breath, and then the animal "dives" again? Functions GET_VARIABLES_LUNG_COLL and GET_VARIABLES_ZIP_NEW(op) have some similar variables that gets updated and GET_VARIABLES_ZIP_NEW(op) is specific for beaked whales so this will also change between species. So we can add a column in the depth-time data file that has 0 for surface/breath and 1 for dive, which I think "dive_flag" is now acccounting for, e.g. dive vs surface

**ChatGPT said:**

Yes—the `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY` are the correct state variables to retain. We should update their lung compartments during surface breathing in the same general way as during diving.

However, I would not represent the data using only:

Plain text

```
0 = surface/breath
1 = dive
```

That combines two different surface states. The model needs to distinguish:

Plain text

```
dive                          submerged
surface between breaths       no fresh atmospheric gas enters
surface breath                tidal volume replaces part of lung gas
```

I recommend adding two columns to the data file:

Plain text

```
dive_flag    breath_flag
```

For example:

Plain text

```
Time,depth,ODBA,locomotion,sonar,dive_flag,breath_flag
26751,0,0.14,8.40,1,0,0
26752,0,0.09,5.63,1,0,0
26753,0,0.08,4.97,1,0,1
26754,0,0.10,5.10,1,0,0
26755,12,0.20,6.00,1,1,0
```

Here:

- `dive_flag = 1` means submerged;
- `dive_flag = 0` means at the surface;
- `breath_flag = 1` marks a breath;
- `breath_flag = 0` means no breath at that time point.

The current `SONAR_FLAG` is not a dive flag. It tracks sonar and dive-response status. Surface versus dive is currently inferred from `P_AMB`, which is calculated from depth.

### Proposed model logic

At each one-second time step:

```matlab
if dive_flag(i) == 1
    % Existing diving model
    % pressure-dependent lung-volume and gas-exchange calculations

elseif breath_flag(i) == 1
    % Surface breath
    % replace part of the lung gas with inspired gas
    % inspired gas: 21% O2, 79% N2

else
    % Surface between breaths
    % keep a finite lung volume
    % allow gas exchange with venous blood
    % no atmospheric gas replacement
end
```

This is better than making the animal briefly “dive” for every breath. A breath does not involve the pressure changes or lung-collapse mechanics associated with a dive. We can still reuse parts of the existing gas-exchange code, but we should not trigger the full diving-pressure algorithm.

### What happens during a breath

At a breath event, the lung gas would be updated using:

- current lung gas in `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY`;
- the assigned tidal volume;
- the available lung volume;
- inspired gas composition;
- the existing animal-specific lung parameters from `GET_VARIABLES_LUNG_COLL_ZIP`;
- the species-specific parameters from `GET_VARIABLES_ZIP_NEW(op)`.

The variables `VT` and `VTmax` should not be reused for tidal volume because they currently describe tracheal/dead-space volume in the lung-collapse model. We should introduce a separate variable, for example:

```matlab
```

```
V_TIDAL
```

Initially, we could assign one fixed tidal volume per species or animal. Later, it could vary with body mass, activity, or surface interval.

### Recommended implementation sequence

1. Keep the successful legacy model unchanged.
2. Copy the working MATLAB code into a new development location.
3. Add `dive_flag` and `breath_flag` as inputs.
4. Create a new function, perhaps:

```
matlab

update_surface_lung_gas.m
```

5. Test the surface-breathing calculation on a very short artificial dataset.
6. Test one real surface interval.
7. Only then run an entire whale record.

Because the current runs take 30 minutes to two hours, we should first develop and test this using a short subset of the data. That will save considerable time and make debugging much easier.

Worked for 48s

---

**You said:**

I am not sure about this approach and the easier may be to just set it to "diving" but to 1 m, so no change in pressure. the only issue here may be that we need to keep track of the dive response and not invoke that but it also has its own flag. what you are proposing may be smarter in the long run, to do it all at the surface, but what i am proposing is more realistic as an animal is almost never at the surface still and they move and even is 30 cm down, this is still underwater. So not sure how to best approach this as we may have to step through the program to figure out the best way

**ChatGPT said:**

I think your proposed approach is reasonable for a first implementation and may be the best way to work with the existing model.

However, simply changing surface depth to 1 m is not quite sufficient. At present, the diving branch models a closed lung exchanging gas with blood. It does not introduce new atmospheric gas. Therefore, we still need a breath event that adds inspired gas to the lung.

A practical hybrid approach would be:

```
Plain text

All time points:
    treat the whale as submerged at shallow depth, for example 0.3–1 m

At a breath:
    replace part of the lung gas using the assigned tidal volume

Between breaths:
    continue using the existing diving gas-exchange calculations

During actual dives:
    continue using the existing pressure-dependent model
```

This would preserve most of the existing code while avoiding the current “infinite surface reservoir” assumption.

The important distinction is:

- `dive_flag` or depth controls ambient pressure and lung mechanics;
- `breath_flag` controls when new inspired gas enters the lung;
- `DIVE_RESPONSE_FLAG` controls the cardiovascular/metabolic dive response.

We should not use the dive-response flag to identify breaths. A breath at 30 cm depth should not automatically invoke `adjust_Q_bradycardia()`.

## Minimal modification

The data could eventually contain:

```
Plain text

Time,depth,ODBA,locomotion,sonar,breath_flag
26751,0,0.14,8.40,1,0
26752,0,0.09,5.63,1,0
26753,0,0.08,4.97,1,1
26754,0,0.10,5.10,1,0
```

Then the model would use:

```matlab
if breath_flag(i) == 1
    % Add inspired gas according to tidal volume
else
    % Continue with existing closed-lung gas exchange
end
```

The breath routine would update the existing arrays:

```matlab
N2_ARRAY(i,5,:)
O2_ARRAY(i,5,:)
CO2_ARRAY(i,5,:)
```

so the rest of the tissue model remains unchanged.

One caution: 1 m is not exactly pressure-free—it represents approximately 1.1 atmospheres absolute—but it is probably small compared with the pressures during the dives. Using 0.3 m would reduce that artificial pressure effect further, although the exact value should be chosen consistently.

The best next step is therefore not yet to rewrite the whole surface model. Instead, we should make a short prototype that:

1. treats the animal as shallowly submerged;
2. adds one artificial breath event;
3. assigns a trial tidal volume;
4. checks how the lung $O_2$, $CO_2$, and $N_2$ arrays change;
5. confirms that the dive-response variables are unaffected.

This will let us step through the existing program and see exactly where the new breath operation should be inserted.

Worked for 44s

---

**You said:**

and how do i copy the code to a new directory? or how to name? and what is a new development location and is this a new directory?

**ChatGPT said:**

A “new development location” means a new folder inside the same repository, while you remain on the `development` branch. We will copy the code, not move it.

I recommend naming the new folder:

```
Plain text

src
```

`src` means “source code.”

Your repository will then look like:

```
Plain text

Gas-dynamics-model/
├── README.md
├── legacy/
│   └── 2026-10-12-original/
├── src/
└── .git/
```

## Create the working copy

In Windows Explorer:

1. Open your local `Gas-dynamics-model` folder.
2. Open:

   ```
   Plain text

   legacy/2026-10-12-original/
   ```

3. Select the MATLAB `.m` files.
4. Right-click and choose **Copy**.
5. Go back to the main `Gas-dynamics-model` folder.
6. Right-click → **New** → **Folder**.
7. Name it:

   ```
   Plain text

   src
   ```

8. Open `src`, right-click, and choose **Paste**.

Do not delete anything from `legacy`.

Initially, place the four CSV files used for testing in `src` as well, because the old loading functions expect the CSV files to be in the current MATLAB folder. We will later improve the loading functions so that the data can be kept in a separate folder.

The temporary structure will therefore be:

```
Plain text

Gas-dynamics-model/
├── legacy/
│   └── 2026-10-12-original/
├── src/
│   ├── driver_to_beakedwhale_tissue_kin.m
│   ├── beakedwhale_tissue_kinetics1.m
│   ├── [other .m files]
│   ├── 11-...data.csv
│   ├── 11-...summary.csv
│   ├── 12-...data.csv
│   └── 12-...summary.csv
└── README.md
```

### Save this change

Open GitHub Desktop. It should show the copied `.m` files as new files.

Use the commit message:

```
Plain text

Create working source copy from legacy model
```

Then choose:

1. **Commit to development**
2. **Push origin**

After this, we will work only on the code in `src`. The files in `legacy` remain the protected reference version.

Worked for 24s

---

**You said:**

i moved only files for the 12th animal, so just two files and edited the driver_fuction. I am not sure I agree with this and let's see if we can agree on how to proceed first: "However, simply changing surface depth to 1 m is not quite sufficient. At present, the diving branch models a closed lung exchanging gas with blood. It does not introduce new atmospheric gas. Therefore, we still need a breath event that adds inspired gas to the lung." I agree that 1 m may be a lot but the "depth" does change continously and the model already tracks this and we could always have the anima; at 0.3 meter and then it surfaces and breathes for a second and then remains at 0.3 meter. Then, I agree that we "refresh" the air inside the lung and we may want to think if we "alter" the lung volume in between breaths, as this may be an important feature, and if so, this will carry through to the lung collapse function

**ChatGPT said:**

I agree with your approach, and my earlier wording was too restrictive. A practical first implementation could be:

Plain text

```
Between breaths:
    whale remains at approximately 0.3 m depth
    existing pressure-dependent model continues to run

During a breath:
    whale is temporarily treated as being at the surface
    a defined tidal volume of inspired gas enters the lung
    the whale then returns to 0.3 m depth
```

This allows us to reuse most of the existing diving code while still representing finite breaths.

The important distinction is that the current surface code does this:

matlab

```
lung gas = fixed atmospheric gas
```

at every surface time point. That is equivalent to completely refreshing the lung from an infinite reservoir. We should change that so that a surface event refreshes only a fraction of the lung, based on tidal volume.

Conceptually:

```matlab
fraction_refreshed = V_TIDAL / V_LUNG_AVAILABLE;
```

Then the new lung gas would be a mixture of:

- the gas already present in the lung;
- newly inspired gas containing 21% oxygen and 79% nitrogen;
- no inspired carbon dioxide.

The existing `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY` would continue to store the resulting lung gas.

One important caution is that `VT` and `VTmax` in the current code refer to tracheal/dead-space volume. We should introduce a separate variable such as:

```matlab
V_TIDAL
```

### Lung volume

I agree that lung volume may need to change during breathing, but we should initially distinguish two effects:

1. `GET_VARIABLES_LUNG_COLL_ZIP` and `N2_lung_coll` describe pressure-dependent lung volume and collapse.
2. Tidal volume describes the volume moved during an individual breath.

At first, I suggest keeping the existing pressure-dependent lung volume model and using tidal volume to determine how much gas is exchanged. Once that works, we can examine whether inhalation and exhalation should also alter `LUNG_VOL` explicitly.

This avoids changing the lung-collapse model and the surface-breathing model simultaneously.

### Flags

I would not use `DIVE_RESPONSE_FLAG` for this. It should continue to control the physiological dive response. A breath at 0.3 m should not automatically invoke bradycardia

or the other dive-response adjustments.

For the prototype, we can add:

```
Plain text

breath_flag
```

A value of `1` would identify the one-second surface/breath event. We could either:

- keep the measured depth unchanged and use `breath_flag` to trigger the breath; or
- temporarily set the depth to zero on breath rows.

I prefer the first option because it preserves the original depth data.

### How we should proceed now

Using whale 12 is a good choice for the prototype. We should:

1. Keep the original model untouched in `legacy`.
2. Keep the two whale-12 input files in `src` for now.
3. Add an artificial `breath_flag` for a short test interval.
4. Modify only the surface branch of `update_lung_gas`.
5. Compare lung \(O_2\), \(CO_2\), and \(N_2\) before and after a breath.
6. Only then run the complete whale record.

So, yes: the “shallow diving with brief surface breaths” approach is a sensible and relatively conservative way to extend the current model.

Worked for 49s

---

**You said:**

But what you call "V_TIDAL" is V_LUNG and we can just use that? Although it is "refilled" each time at the surface, the alveolar gas values "assures" that there is a level of dead space ventilation, although this is misrepresetned as the animal can never hyper or hypoventilate. We may want to consider this in the model, or it may become a property if we we use VLmax in lung collapse as then we can "mix" a tidal volume with the last lung gas volume and gas content?

**ChatGPT said:**

Yes—your interpretation is useful, but I would separate the terminology carefully.

In the current code:

- `V_LUNG` / `VLmax` represents the available or maximum lung volume.
- `VTmax` represents tracheal/dead-space volume.
- Neither is explicitly the volume exchanged during one breath.

If we set the breath volume equal to `V_LUNG` or `VLmax`, we are making the explicit assumption that each breath completely refreshes the available lung volume. That would reproduce something close to the current model, but as a discrete breath rather than as continuous atmospheric equilibration.

Your point is correct: the current use of `ALV_SURFACE_VALUE` imposes a fixed alveolar gas composition and therefore hides hyperventilation and hypoventilation. It does not explicitly calculate dead-space ventilation; it effectively imposes an ideal alveolar boundary condition.

A better formulation would be:

matlab

```
V_LUNG       % available lung volume at the current pressure
V_BREATH     % volume exchanged during one breath
V_DEADSPACE  % tracheal/dead-space volume
```

For an initial prototype, we could define:

matlab

```
V_BREATH = VLmax;
```

This would mean complete replacement of the lung gas at each breath.

The more flexible version would use:

matlab

```
V_BREATH = fraction * VLmax;
```

Then the inspired gas would be mixed with the gas already present:

matlab

```
fraction_refreshed = V_BREATH / V_LUNG;

n_new = (1 - fraction_refreshed) .* n_old ...
      + fraction_refreshed .* n_inspired;
```

This would allow:

- smaller breaths → hypoventilation;
- larger breaths → hyperventilation;
- different breathing intervals → different minute ventilation;
- a species-specific or animal-specific `V_BREATH`.

I would therefore suggest two stages:

1. First test the model with `V_BREATH = VLmax`, which approximates full lung replacement and lets us verify the mechanics.
2. Then reduce `V_BREATH` to a defined fraction of `VLmax`, allowing the model to represent partial refresh and altered ventilation.

I would initially leave the lung-collapse function unchanged. Let it determine the pressure-dependent available lung volume, while the breath routine determines how much gas is exchanged within that volume. Later, if necessary, we can model the expansion and contraction of the lung during the breath itself.

So the key change is that `ALV_SURFACE_VALUE` should become the composition of inspired gas, rather than directly replacing the entire `O2_ARRAY`, `CO2_ARRAY`, and `N2_ARRAY` lung compartments at every surface time point.

Worked for 37s

---

**You said:**

V_LUNG is not necessarily equal to VLmax but it can be smaller, but yes if V_LUNG=VLmax, then the entire lung volume, is exchanged. In your definition, I would then set V_BREATH=V_LUNG and keep VLmax equal to total lung capacity. Then we set V_LUNG=VLmax*fraction and V_BREATH=V_LUNG and fraction of total lung capacity would be V_BREATH/VLmax. We need to make sure we keep some of these unchanged as they are used in the model throughout and especially in the lung collapse portion these variables are required

Worked for 48s >

**ChatGPT said:**

Yes, I understand your definition now, and conceptually it is coherent:

```matlab
VLmax   = total lung capacity
V_LUNG  = functional lung volume used for a breath
V_BREATH = V_LUNG
```

Then, for example:

```matlab
V_LUNG = VLmax * fraction;
V_BREATH = V_LUNG;
```

If `fraction = 1`, the whole lung volume is exchanged during a breath. If `fraction < 1`, only part of the total lung capacity is exchanged.

The important caution is that, in the current code, `V_LUNG` is not used only as a breath volume. It is also used to calculate:

```matlab
VLinit
VLinit0
ALV_SURFACE_VALUE
n_N2_LUNG
n_O2_LUNG
n_CO2_LUNG
```

and these values feed into the lung-collapse calculations. Therefore, changing `V_LUNG` will indeed affect the lung volume and gas stores throughout the model. That may be exactly what we want, but it must be done deliberately.

The current code also has some overlapping terminology:

- `V_LUNG` is initialized as the total starting lung volume.
- `VLinit` is the initial alveolar volume after subtracting `VTmax`.
- `VLmax` is the maximum volume used by the lung-collapse equation.
- `VTmax` is the tracheal/dead-space volume.

So before changing the code, we should document the intended relationship explicitly:

matlab

```
VLmax = total lung capacity at the relevant reference condition
VTmax = tracheal/dead-space volume
V_LUNG = functional lung volume available for the breath
VLinit = alveolar volume at the beginning of a dive
V_BREATH = volume exchanged during the breath
```

For the first prototype, I suggest:

matlab

```
V_BREATH = V_LUNG;
```

but keep the existing `V_LUNG` calculation unchanged. This allows us to test the new breath exchange without simultaneously changing the lung-collapse initialization.

Once that works, we can test:

matlab

```
V_LUNG = VLmax * 0.8;
V_BREATH = V_LUNG;
```

or another fraction and observe how this affects:

- initial lung gas stores;
- alveolar volume;
- lung collapse;
- nitrogen uptake;
- oxygen and carbon dioxide exchange.

Thus, I agree with your physical concept. The next step should be to make a small variable map and then implement the breath exchange while preserving the current lung-collapse variables. Your whale-12 prototype is the right place to do this.

**You said:**

we also must remember that VLmax is made up of two parts, dead space and alveolar space, so the variable TAC is the total aveolar capacity, this is another odd name variable but to keep with an old paper. For an animal that hold its breath, maybe these different air spaces/volumes are mixed so they are not problematic like in a human, in

that dead space ventilation is basically wasted volume and breathing. OK, so let's get started and what do I do first? I have moved the code to the SRC folder

**ChatGPT said:**

Good clarification. We should use the following definitions:

```
Plain text

VLmax  = total respiratory capacity
VTmax  = dead-space/tracheal volume
TAC    = total alveolar capacity = VLmax - VTmax
V_LUNG = operational lung volume used by the model
```

The existing model already treats the gas stores in `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY` largely as a mixed lung compartment, while `LUNG_VOL` tracks alveolar and tracheal volumes. That is a reasonable starting point for a diving animal. We do not need to create a separate dead-space gas compartment immediately.

## First thing to do now

Before making any model changes, save the current development setup.

In GitHub Desktop:

1. Confirm that the current branch is `development`.
2. Review the changed files.
3. Select the MATLAB files in `src` and the updated driver.
4. Do not select the large CSV files or generated output files.
5. Commit with:

```
Plain text

Create development source copy for whale 12 prototype
```

6. Push the commit to GitHub.

This gives us a recorded starting point for the new model.

## Then upload the current `src` code here

Please create a ZIP of:

```
Plain text
```

```
Gas-dynamics-model/src/
```

You can omit the CSV files if the ZIP becomes large. I already have the whale-12 data files and the original legacy code, but I need to see your current `src` version, especially the edited driver.

Do not run the full model again yet.

Once I have the current `src` code, the first technical step will be to document and verify where these variables are initialized and used:

```matlab
V_LUNG
VLmax
VTmax
TAC
VLinit
VLinit0
LUNG_VOL
```

Then we will create a small test of one breath using the existing lung-gas arrays before changing the full simulation. The initial test will use:

```matlab
V_BREATH = V_LUNG;
```

and will let us see whether the whole available lung volume is refreshed correctly while preserving the lung-collapse calculations.

Worked for 43s

---

**You said:**

what am i supposed to do in github, am a bit lost i copied the files to the src in explorer and now what

**ChatGPT said:**

You are now at the point where you save the copied files as a commit on the `development` branch.

### In GitHub Desktop

1. Open **GitHub Desktop**.
2. Select the repository:

```
Plain text

Gas-dynamics-model
```

3. At the top, confirm:

```
Plain text

Current branch: development
```

4. Click the **Changes** tab on the left.

You should see the MATLAB files you copied into `src`.

### Select the correct files

Select the new `.m` files inside:

```
Plain text

src/
```

Do not select:

- the large CSV files;
- generated `.mat` or output files;
- anything inside `legacy/`.

If GitHub Desktop shows a changed file inside `legacy`, stop before committing and tell me, because the legacy copy should remain unchanged.

### Commit the source copy

At the bottom-left, enter:

```
Plain text

Create development source copy for whale 12 prototype
```

Then click:

Plain text

```
Commit to development
```

After the commit finishes, click:

Plain text

```
Push origin
```

This uploads the `src` folder to the `development` branch on GitHub. It does not change `main` or the legacy tag.

After pushing, open GitHub in your browser, select the `development` branch, and you should see:

Plain text

```
src/
legacy/
README.md
```

Do not make any further code changes yet. Once this is done, we can inspect the current `src` copy and begin the small breath-exchange prototype.

Worked for 16s

---

**You said:**

ok, ready to check src copy

**ChatGPT said:**

Great. Please create and upload a ZIP of the current:

Plain text

```
Gas-dynamics-model/src/
```

Include all `.m` files, including the edited driver. You can omit the large CSV files because I already have the whale-12 data.

In Explorer:

1. Right-click the `src` folder.
2. Choose **Send to → Compressed (zipped) folder**.
3. Upload the resulting ZIP here.

I will then check that:

- all required MATLAB functions are present;
- the edited driver points to the correct files;
- the legacy code remains separate;
- `V_LUNG`, `VLmax`, `VTmax`, `TAC`, `VLinit`, and `LUNG_VOL` are used consistently;
- we can safely begin the surface-breathing prototype.

**You said:**

**All files.zip**
File

do you want or not the whale-12 data in the src folder?

Worked for 23s >

**ChatGPT said:**

For the current setup:

- Keep the whale-12 CSV files in your local `src` folder so MATLAB can run the code unchanged.
- Do not commit the CSV files to GitHub yet.
- They do not need to be included in the ZIP for me to inspect the MATLAB code because I already have the whale-12 data from your earlier upload.

I inspected the new `All files.zip`. It contains the MATLAB code, including your edited driver and modified `GET_VARIABLES_LUNG_COLL.m`, but it does not contain the whale-12 CSV files. It also contains:

Plain text

```
SONAR_FLAG
```

That is a generated output file, so it should also not be committed to GitHub.

For now, your local `src` folder should contain:

Plain text

```
src/
├── all MATLAB .m files
├── 11-...csv files, if needed
├── 12-...csv files
└── generated output files, locally only
```

In GitHub Desktop, commit only the MATLAB source files. Leave the CSV files and `SONAR_FLAG` unchecked.

The important thing is that your local `src` folder contains the data for MATLAB, while GitHub stores the reproducible source code. Later we can create a separate local `data` folder and modify the loading functions to use it properly.

**You said:**

i removed the sonar_flag data and also an onlo oncluding 12-whale not the 11th as the code has been updated and the loop only uses 12.

**ChatGPT said:**

That is correct for the current prototype.

Your `src` folder should now contain:

- all MATLAB `.m` files;
- the whale-12 data CSV;
- the whale-12 summary CSV;
- the updated driver that loops only over whale 12.

It does not need:

- whale-11 data;
- `SONAR_FLAG`, because that is generated during the run;
- old output files.

In GitHub Desktop, verify that the changes are only in `src`, not in `legacy`. Then commit the MATLAB changes with:

Plain text

```
Prepare whale-12 development prototype
```

Push that commit to `development`.

Do not commit the whale-12 CSV files or generated output files. Once that is done, the code structure is ready for us to begin examining the surface-breathing modification.

Worked for 16s

**You said:**

ok, done and what should we make now, where do we define the new varaibles or is it just in update lung gas?

Worked for 59s >

**ChatGPT said:**

The new variables should be initialized in the setup functions and used by `update_lung_gas`; they should not be defined only inside `update_lung_gas`.

I also noticed an important issue in the current `src` copy: the whale-12 model calls:

```matlab
GET_VARIABLES_LUNG_COLL_ZIP(...)
```

from `GET_VARIABLES_ZIP_NEW.m`.

Therefore, changes made to:

```matlab
GET_VARIABLES_LUNG_COLL.m
```

will not affect the current whale-12 model. The corresponding beaked-whale file is:

```matlab
GET_VARIABLES_LUNG_COLL_ZIP.m
```

The two files should remain separate because they represent different model versions/species.

### Where the variables belong

`GET_VARIABLES_ZIP_NEW.m` should define species-specific quantities such as:

matlab

```
V_LUNG
BREATH_FRACTION
```

`GET_VARIABLES_LUNG_COLL_ZIP.m` should define the lung geometry:

matlab

```
VLmax
VTmax
TAC
VLinit
VLinit0
```

Then we can define:

matlab

```
V_BREATH = V_LUNG;
```

after `V_LUNG` has been initialized.

`update_lung_gas.m` should only use these variables and update the lung gas arrays:

matlab

```
N2_ARRAY
O2_ARRAY
CO2_ARRAY
```

A useful initial variable map would be:

matlab

```
VLmax       % total lung capacity
VTmax       % tracheal/dead-space volume
TAC         % alveolar capacity: VLmax - VTmax
V_LUNG      % functional lung volume used by this animal
V_BREATH    % volume exchanged during a breath
VLinit      % current initial alveolar volume for lung-collapse calculations
VLinit0     % reset value of VLinit
LUNG_VOL    % time-dependent alveolar/tracheal volumes
```

For the first prototype:

matlab

```
V_BREATH = V_LUNG;
```

Later, if we want partial ventilation:

matlab

```
V_LUNG = VLmax * BREATH_FRACTION;
V_BREATH = V_LUNG;
```

But we should not change `V_LUNG` yet, because it is already used to initialize the lung gas stores and `VLinit0`.

So the next step is:

1. Leave the non-ZIP file alone for now.
2. Confirm that `GET_VARIABLES_LUNG_COLL_ZIP.m` contains the beaked-whale values we intend to use.
3. Add the new breath variables there or in `GET_VARIABLES_ZIP_NEW.m`.
4. Only then modify `update_lung_gas.m`.

We should make these changes in `src`, never in `legacy`.

**You said:**

OK, SO I removed the older version of GET_VARIABLES_LUNG_COLL.m and now have these open. But assume we need V_BREATH in both but in one, GET_VARIABLES_LUNG_COLL_ZIP.m, it is only used once for initilization. So do we keep with the "bad practice" of using these variables as global?

**ChatGPT said:**

For this first modification, I would temporarily continue using the existing global-variable structure. Refactoring all globals now would create a much larger change and make it harder to determine whether any new result comes from the breathing model or from a general code rewrite.

The important rule is:

> Use the global variable for compatibility, but define it in one authoritative place.

Do not assign different values to `V_BREATH` independently in both files.

For the current beaked-whale model, the best arrangement is:

matlab

```
GET_VARIABLES_ZIP_NEW.m
```

sets the animal-specific value of `V_LUNG` and the breathing fraction, while:

matlab

```
GET_VARIABLES_LUNG_COLL_ZIP.m
```

sets `VLmax`, `VTmax`, and `TAC`.

Then, after `GET_VARIABLES_LUNG_COLL_ZIP` has been called, define:

matlab

```
V_BREATH = V_LUNG;
```

For example, in `GET_VARIABLES_ZIP_NEW.m`:

matlab

```
global V_LUNG V_BREATH BREATH_FRACTION
```

Set the fraction:

matlab

```
BREATH_FRACTION = 1.0;
```

Then, after this existing call near the end of the function:

matlab

```
GET_VARIABLES_LUNG_COLL_ZIP(n_O2_LUNG,n_CO2_LUNG,n_N2_LUNG);
```

add:

```matlab
V_BREATH = V_LUNG * BREATH_FRACTION;
```

For the first prototype, this gives:

```matlab
V_BREATH = V_LUNG;
```

Later, if we want partial ventilation:

```matlab
BREATH_FRACTION = 0.8;
V_BREATH = V_LUNG * BREATH_FRACTION;
```

Any function that uses the value, such as `update_lung_gas.m`, would declare:

```matlab
global V_BREATH
```

The lung-collapse function does not need to define `V_BREATH` unless it directly uses it. It should continue defining:

```matlab
VLmax
VTmax
TAC
VLinit
VLinit0
```

This preserves the existing lung-collapse calculations.

Eventually, the cleaner design would be to replace the globals with a parameter structure, for example:

```matlab
```

```
params.VLmax
params.VTmax
params.TAC
params.V_LUNG
params.V_BREATH
```

But for now, adding a small number of documented globals is the safest incremental approach. We should not duplicate the assignment of `V_BREATH` in both setup files.

Worked for 37s

---

**You said:**

Sorry, but your logic is a both contradictory and goes around in circles. I agree that we should replace global with structures, but that is for the future. Now we want this to work and I have added V_LUNG V_BREATH and BREATH_FRACTION as globals in GET_VARIABLES_ZIP_NEW abd then here:
M_B=op%send the animal specific body mass to function 200;%animals body mass, from andrews-jeb-200-2083-2095-1997, Seal: GJ325, Mb=177kg, Mb=104kg Kooyman et al, 1972 AmJP paper
V_LUNG=25*M_B/1000%25 ml/kg from Zimmer et al., 2007, Miller et al., 2016
%V_LUNG= 0.135*M_B^0.92;%Max volume defined as 50.4 ml/kg for elephant seal in Kooyman et al. AmJP 1972 assumed to be close to max insp volume, 20.4 ml/kg was seen during chamber dives
BREATH_FRACTION = 1.0; and after the call to the:
GET_VARIABLES_LUNG_COLL_ZIP(n_O2_LUNG,n_CO2_LUNG, n_N2_LUNG);
V_BREATH = V_LUNG * BREATH_FRACTION;

**ChatGPT said:**

You are right. Your implementation is coherent, and my previous replies mixed two alternative designs. Sorry for the confusion.

What you have now is correct for the first prototype:

```matlab
```

```
global V_LUNG V_BREATH BREATH_FRACTION
```

Then:

matlab

```
M_B=op;
V_LUNG=25*M_B/1000;

BREATH_FRACTION = 1.0;
```

After:

matlab

```
GET_VARIABLES_LUNG_COLL_ZIP(n_O2_LUNG,n_CO2_LUNG,n_N2_LUNG);
```

you add:

matlab

```
V_BREATH = V_LUNG * BREATH_FRACTION;
```

That means:

Plain text

```
BREATH_FRACTION = 1
V_BREATH = V_LUNG
```

so the first prototype exchanges the whole functional lung volume during a breath.

For whale 12, with `M_B = 2000 kg`, the expected values are approximately:

Plain text

```
V_LUNG       = 50.000 L
VLmax        = 50.000 L
VTmax        = 3.335 L
TAC          = 46.665 L
V_BREATH     = 50.000 L
```

The important point is that `GET_VARIABLES_LUNG_COLL_ZIP.m` should continue defining:

```matlab
VLmax
VTmax
TAC
VLinit
VLinit0
```

It does not need to assign `V_BREATH` itself.

Now add the following global declaration to `update_lung_gas.m`:

```matlab
global V_LUNG V_BREATH BREATH_FRACTION
```

The variables must be declared `global` separately in every function that uses them.

Before running the full 30-minute model, perform a short initialization check in MATLAB:

```matlab
clearvars
GET_VARIABLES_ZIP_NEW(2000)

global V_LUNG V_BREATH BREATH_FRACTION
global VLmax VTmax TAC VLinit VLinit0

[V_LUNG, V_BREATH, BREATH_FRACTION, ...
 VLmax, VTmax, TAC, VLinit, VLinit0]
```

This should return values close to those above.

At this stage, `V_BREATH` is defined but not yet used to alter the lung gas. That is deliberate. The next code change will be in `update_lung_gas.m`, where we add the breath-specific gas update while leaving the diving branch unchanged.

Worked for 1m 22s

---

**You said:**

nice and the check worked

**ChatGPT said:**

Excellent. The initialization is now working, so we should take the next small step without changing the full model yet.

The next thing to create is a separate function:

Plain text

```
surface_breath_exchange.m
```

This function will only calculate how the existing lung gas is mixed with inspired gas during one breath. It will not initialize `V_LUNG`, `VLmax`, `TAC`, or any lung-collapse variables.

Its inputs should be:

matlab

```
current lung gas
current lung volume
V_BREATH
surface pressure
gas constant
inspired gas fractions
```

Its output will be the updated amounts of:

matlab

```
N2
O2
CO2
```

Conceptually:

matlab

```
refresh_fraction = V_BREATH / current_lung_volume;
```

Then:

```matlab
new gas = remaining old gas + inspired gas entering during the breath
```

For the first test, because:

```matlab
V_BREATH = V_LUNG
```

and the lung volume is also approximately `V_LUNG`, the refresh fraction will be 1. The lung gas should therefore become the inspired gas mixture. That gives us a simple test against the current `ALV_SURFACE_VALUE` behavior.

The new function should not use globals internally. It can receive the necessary values as inputs. The existing model can continue using globals while we gradually add this new functionality.

After this helper works, we will:

1. add a temporary breath event;
2. call the helper from `update_lung_gas.m`;
3. update `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY`;
4. compare the result with the current surface-reset behavior.

For now, do not run the full whale model again. The next change should be the small standalone breath-mixing function, which we can test in seconds.

Worked for 1m 12s

---

Yesterday 11:54 AM

**You said:**

morning, so what is next. Should I type out the code? I am still a bit rusty how different arrays and functions are linked, so maybe I should read through the old code?

**ChatGPT said:**

Good morning. The next step is to understand the existing flow before changing it. You do not need to read all 60 files—just the files controlling the lung gas update.

Open these files in `src`:

1. `driver_to_beakedwhale_tissue_kin.m`
2. `beakedwhale_tissue_kinetics1.m`
3. `GET_VARIABLES_ZIP_NEW.m`
4. `update_lung_gas.m`
5. `GET_VARIABLES_LUNG_COLL_ZIP.m`
6. The functions called by `update_lung_gas`, especially the `update_*_case_*` functions.

As you read, trace four things:

- Where `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY` are created.
- Which element corresponds to the current time step.
- Where `update_lung_gas` is called.
- Which function changes the lung gas after each time step.

The important concept is:

matlab

```
previous lung gas
        ↓
update_lung_gas
        ↓
new lung gas
        ↓
N2_ARRAY/O2_ARRAY/CO2_ARRAY
        ↓
next time step
```

For now, do not modify `update_lung_gas`. We should first create and test a small independent function called:

Plain text

```
surface_breath_exchange.m
```

This function will not affect the model yet. It will simply test the proposed breath-mixing calculation.

Create `src/surface_breath_exchange.m` with:

matlab

```
function n_new = surface_breath_exchange( ...
    n_old, V_gas, V_breath, P_inspired, RT, inspired_fraction)
%SURFACE_BREATH_EXCHANGE Mix inspired gas into the existing lung gas.
%
% n_old            1x3 vector: [N2 O2 CO2] moles before the breath
% V_gas            gas-space volume
% V_breath         volume exchanged during the breath
% P_inspired       inspired gas pressure
% RT               gas constant multiplied by absolute temperature
% inspired_fraction 1x3 vector: [fraction N2, fraction O2, fraction CO2]
%
% n_new             1x3 vector: [N2 O2 CO2] moles after the breath

    if V_gas <= 0
        error('V_gas must be greater than zero.');
    end

    if V_breath < 0
```

Then run this small test in MATLAB:

matlab

```
n_old = [0.01 0.01 0.001];

V_gas = 1.0;
V_breath = 1.0;
P_inspired = 1.0;
RT = 0.082057 * 310;

inspired_fraction = [0.79 0.21 0.00];

n_new = surface_breath_exchange( ...
    n_old, V_gas, V_breath, P_inspired, RT, inspired_fraction)

expected = inspired_fraction .* P_inspired .* V_gas ./ RT
```

Because `V_breath = V_gas`, `n_new` and `expected` should be identical. That confirms a complete lung refresh works.

Then test partial exchange:

matlab

```
V_breath = 0.25;

n_partial = surface_breath_exchange( ...
    n_old, V_gas, V_breath, P_inspired, RT, inspired_fraction)
```

This should produce values between the old gas and the inspired gas.

The function is deliberately separate from the old model at this stage. Once you understand the array flow and this test works, we will connect it to `update_lung_gas` at the precise point where a new `breath_flag` is detected. This keeps the first model change small and reversible.

Yesterday 2:09 PM

**You said:**

ok, so this works and creates an exchange or fraction, but the V_breath=0.25, why now all of a sudden does the last output have a non-zero value as if you update V_breath to be a fraction of V_breath now it has changed the value, but 0.25 of 0 is 0? Also, this may work fine but how will we incorporate this into Update_lung_gas? In that function, we have 3 cases, or actually A, B, C, D, E, F, and G. For A the animal is at the surface at time -1 and 0, For B at the surface at t-1 but underwater at 0, for C undrewater at t-1 and at the surface at 0. For D, E, and F, the animal is underwater at time t-1 and 0. For the cases B, D, E and F, the function interation_GAS_lung_search is used for each gas. So we will likely need to use the iteration_GAS_lung_search for these other cases and then use this new function for cases A, B and C. But if we change the "SURFACE_VALUE" so it is always at depth except for breaths, we can use the iteration_GAS_lung_search fundciton for all cases except during a rapid breath and just have one case? Or am i jumping ahead?

**ChatGPT said:**

You are seeing the expected result. `V_breath = 0.25` is a volume, not a gas fraction. In the test:

matlab

```
V_gas = 1.0;
V_breath = 0.25;
```

the exchanged fraction is:

```matlab
V_breath / V_gas = 0.25
```

Therefore:

```matlab
n_new = 0.75*n_old + 0.25*n_inspired;
```

For $CO_2$, inspired gas contains zero $CO_2$, so:

```matlab
n_new_CO2 = 0.75*n_old_CO2;
```

It remains non-zero because only 25% of the gas was replaced. For a complete refresh:

```matlab
V_breath = V_gas;
```

then:

```matlab
n_new = n_inspired;
```

and $CO_2$ becomes zero, assuming inspired $CO_2$ is zero.

The important distinction is:

| Variable | Meaning |
| --- | --- |
| `V_breath` | Actual volume exchanged |
| `V_gas` | Gas volume available for mixing |
| `V_breath/V_gas` | Fraction of the gas replaced |

Regarding `update_lung_gas`: you are not jumping ahead conceptually, but I would not remove the existing A–G cases yet. The safest first implementation is to add a breath event as a separate condition before the existing logic:

matlab

```
if breath_flag == 1

    % Apply atmospheric breath mixing

else

    % Existing cases A-G remain unchanged

end
```

That gives us:

Plain text

```
breath event?
     |
     ├── yes → mix inspired gas into lung gas
     |
     └── no  → use existing A-G pressure-dependent logic
```

This is preferable initially because the A–G cases describe pressure transitions, whereas a breath event describes gas replacement. They are related, but they are not exactly the same process.

Your proposed simplification may eventually be possible:

- keep the animal at shallow depth between breaths;
- use `iteration_GAS_lung_search` during those periods;
- use the new breath-mixing function only during a breath;
- avoid treating ordinary shallow movement as a transition to the atmospheric reservoir.

However, the existing A–G code may still contain important resets of `VLinit`, `VLprev`, `Pint`, and other lung-collapse variables. Removing those cases before checking their purpose could subtly change lung mechanics.

So the next step should be:

1. Keep A–G unchanged.
2. Add a temporary `breath_flag` input.

3. Add a breath branch at the top of `update_lung_gas`.
4. Initially use `V_BREATH = V_LUNG`, meaning a complete refresh.
5. Run one short test and compare lung gas immediately before and after the breath.
6. Only then test partial breaths.

One important question for the integration is what we use as `V_gas`. For the first prototype, we should explicitly decide whether the breath mixes with:

```matlab
V_gas = V_LUNG
```

or with the available alveolar gas volume:

```matlab
V_gas = V_LUNG - VTmax
```

Given your definition of `VTmax` as dead-space volume, the second option is physiologically more explicit. But because the current model initializes and stores lung gases using `V_LUNG`, I recommend testing with `V_gas = V_LUNG` first, then checking whether the existing `N2_ARRAY`, `O2_ARRAY`, and `CO2_ARRAY` are defined for total lung volume or alveolar volume before changing it.

Worked for 31s

---

**You said:**

The issue is  that cases A and C does not assume a finite lung reservoir so we will have to "fix" them no matter what and by "merging them" into the other cases, we now get rid of all of that. The "update_GAS_case_A_or_C" does not update VLinit, VLprev or Pint, it just tkes the predetermined alveolar gas values from ALV_SURFACE_VALUE defined in GET_VARIABLES_ZIP_NEW and gives it that many moles of gas. Basically, at the surface PCO2=42 mmHg, PO2=116mmHg or PN2 =whatever it is in mmhg at the surface. So these have to go anyways and could be defined at time step 1, and then it should be updated through the new equation you just wrote. So I think we need to think about this before writing new code?

**ChatGPT said:**

Yes—you are right. We should pause before adding code. The previous `update_GAS_case_A_or_C` logic is not compatible with the finite-reservoir model because it resets the lung gas to the predetermined `ALV_SURFACE_VALUE` whenever the animal is at the surface.

That means these values:

```matlab
PCO2 = 42 mmHg
PO2  = 116 mmHg
PN2  = remaining pressure
```

are currently imposed values, not values emerging from the previous lung gas plus a finite breath. They should therefore be used only to initialize the model at the first time step, not repeatedly during the simulation.

The revised logic should be:

```
Initial time step
    └── initialize lung gas using ALV_SURFACE_VALUE

Every later time step
    |
    ├── breath_flag = 1
    |       └── exchange a finite breath with inspired air
    |
    └── breath_flag = 0
            └── use the existing finite-reservoir gas iteration
                while accounting for the current pressure
```

Thus, cases A–G should no longer determine whether the lung is connected to an infinite atmospheric reservoir. They only describe the pressure/depth transition:

| Situation | New treatment |
|---|---|
| A: surface → surface | finite lung iteration unless a breath occurs |
| B: surface → underwater | finite lung iteration |
| C: underwater → surface | finite lung iteration unless a breath occurs |

| Situation | New treatment |
| --- | --- |
| D–G: underwater transitions | existing finite lung iteration |

`SURFACE_VALUE` can still be retained as a pressure value for the mechanical calculations, but it should no longer trigger:

```matlab
n_N2 = ALV_SURFACE_VALUE(...)
n_O2 = ALV_SURFACE_VALUE(...)
n_CO2 = ALV_SURFACE_VALUE(...)
```

The crucial distinction is:

- `ALV_SURFACE_VALUE`: initial condition or reference state.
- `breath_flag`: actual atmospheric gas exchange.
- `iteration_GAS_lung_search`: changes in the closed, finite lung reservoir between breaths.

So the new breath event should be treated as an additional process, not as another surface case:

```matlab
if breath_flag == 1
    % finite breath exchange with inspired gas
else
    % finite-reservoir gas update using the existing iteration
end
```

However, before writing this branch, we need to inspect two things carefully:

1. Whether `iteration_GAS_lung_search` can operate when the animal moves from underwater to shallow/surface pressure without relying on the old A/C reset.
2. What should happen to `VLinit`, `VLprev`, and `Pint` immediately after a breath. The old A/C code did not update them, but that does not necessarily mean the new breath branch can ignore them. We need to ensure that the gas composition and the lung-collapse state remain internally consistent.

There is also an important physiological point: if we set

```matlab
V_BREATH = V_LUNG
```

and replace the entire lung volume with inspired gas, the immediate post-breath gas will have approximately 21% oxygen, 79% nitrogen, and zero $CO_2$. That is different from the current imposed alveolar surface values. $CO_2$ and the lower alveolar oxygen level would then need to develop through gas exchange with blood and metabolism during the following time steps.

Therefore, the next step should not be coding yet. Open:

- `update_lung_gas.m`
- `iteration_GAS_lung_search.m`
- `update_GAS_case_A_or_C.m`

We should identify exactly which variables A/C reset, which variables the iteration expects, and whether the iteration itself can replace A–G as the general no-breath pathway. Then we can design the new branch correctly rather than simply inserting the mixing function into the existing structure.

Worked for 37s

---

Yesterday 3:38 PM

**You said:**

We may need to consider how to treat B and that may have to also be finite lung iteration unless a breath occurs. Maybe surface_value can look at the breath indicator instead, or maybe just create a new one instead and leave this in the for, elseif and else statements? Then use "breath_flag and decide how long of a time step we need for the ventilation? For interation_GAS_lung_search, there are 2 cases, one assumes that there is perfect equilibrium between lung and blood and that the partial pressure at the end of the pulmonary capillary is equal. The other uses the shunt value and then alters the level. This same should work even if we are at 0.3 or 0.,1 meter where lung compression is negligible. Pint should be updated as before if we are not at surface so by removing case A and C and just inserting one that I was just at the surface for a breath and reinitiating VLinit, then things should be fine. Even if the animal is at the surface but not breahting, then the lung volume should remain the same apart from changes due to exchange of gases, which is done by the

interation_GAS_lung_search. I cant remember where VLprev and VLinit are updated though, I think that VLinit is just used once. Can you scan and check in whhat functions they are used and potentially updated?

Worked for 1m 1s >

**ChatGPT said:**

I scanned the code. Your interpretation is largely correct, but I found one additional important surface reset.

## Where the variables are used

| Variable | Initialized | Updated later |
|---|---|---|
| `VLinit` | `GET_VARIABLES_LUNG_COLL_ZIP.m` | `update_lung_gas.m` |
| `VLprev` | `GET_VARIABLES_LUNG_COLL_ZIP.m` | `N2_lung_coll.m` |
| `Pint` | `GET_VARIABLES_LUNG_COLL_ZIP.m` | `N2_lung_coll.m` |
| `VLinit0` | `GET_VARIABLES_LUNG_COLL_ZIP.m` | Not normally changed |
| `VLprev0` | `GET_VARIABLES_LUNG_COLL_ZIP.m` | Not normally changed |

The active chain is:

Plain text

```
update_lung_gas
    └── est_va
          └── N2_lung_coll
```

## What each variable actually does

`VLinit` is not used only once. In `update_lung_gas.m`, it is modified according to changes in total lung gas:

```matlab
VLinit = VLinit0 * currentvalue / initialvalue;
```

This allows subsequent lung-collapse calculations to account for gas lost from or added to the lung.

`VLprev` is updated in `N2_lung_coll.m`:

```matlab
VLprev = VL;
```

It is used as the starting estimate for the next `fzero` search.

`Pint` is also updated in `N2_lung_coll.m`:

```matlab
Pint = Pint_init * (VLinit + VTinit) ./ (VL + VT);
```

So `VLprev` and `Pint` are mechanical state variables updated through `N2_lung_coll`, whereas `VLinit` is partly a gas-content-dependent state variable updated in `update_lung_gas`.

### The additional problem: `est_va.m`

Even if we remove cases A and C from `update_lung_gas`, `est_va.m` currently resets the lung volume whenever the animal is at `SURFACE_VALUE`:

```matlab
if P_AMB(i) == SURFACE_VALUE
    LUNG_VOL(i,1) = VLinit0;
    LUNG_VOL(i,2) = VTinit;
    va = VLinit0;
else
    va = N2_lung_coll(P_AMB(i)*101.3,i);
end
```

Therefore, surface no-breath periods are currently reset in two places:

1. `update_lung_gas.m` resets gas composition using `ALV_SURFACE_VALUE`.
2. `est_va.m` resets lung volume to `VLinit0` and `VTinit`.

That second reset is easy to miss.

### Revised conceptual structure

Your proposed approach should be:

```
Plain text

At initialisation:
    use ALV_SURFACE_VALUE once

At every later time step:
    determine whether a breath occurred

    if breath occurred:
        replace/mix finite inspired volume
        reset the mechanical starting state appropriately

    else:
        use the existing finite-reservoir gas iteration
        do not reset gas to ALV_SURFACE_VALUE
        do not reset lung volume merely because pressure is SURFACE_VALUE
```

This means that `SURFACE_VALUE` should describe pressure, not breathing. A surface point without a breath should be treated as an ordinary finite-reservoir time step.

### How I would handle the existing cases

Initially, I would leave the existing `for/elseif/else` structure in place while we modify its meaning:

- Case A: replace the infinite-reservoir update with finite iteration unless `breath_flag = 1`.
- Case B: finite iteration unless `breath_flag = 1`.
- Case C: finite iteration unless `breath_flag = 1`.
- Cases D–G: retain the existing finite iteration.

In practice, the clean final structure may become:

```
matlab

if breath_flag == 1

    % finite breath exchange
    % reset/reinitialize mechanical state as appropriate

else

    % finite gas exchange for all pressure/depth states
```

```
    % including surface-to-surface and underwater-to-surface

end
```

But before merging the cases, we need to confirm that `iteration_*_lung_search` works when `Pamb` is at surface pressure. The gas-iteration functions themselves do not impose an infinite atmospheric reservoir; they exchange gas between the lung and blood using the current lung volume and shunt. That supports your idea that they should work at 0.1–0.3 m or at the surface.

The main remaining design issue is the surface volume calculation in `est_va.m`. We need to decide whether, in a surface no-breath step, it should:

1. retain the previous `LUNG_VOL` values, or
2. call `N2_lung_coll` at surface pressure, or
3. use a special breath-dependent reset only when `breath_flag = 1`.

My recommendation is option 3:

- no breath: retain/use the finite mechanical state;
- breath: explicitly reset the lung to the chosen post-breath condition;
- `ALV_SURFACE_VALUE`: initialization only.

So the next thing to inspect is the order in which `est_va`, `update_lung_gas`, and `N2_lung_coll` are called inside the time-step loop. That order determines exactly where the breath event should be inserted.

Yesterday 5:50 PM

**You said:**

correct, we have update_lung_gas.m using the updated alv_surface_values but I think we can use the old values from assuming perfect mixing of the entire lung, and then "mix" whatever volume is left (concetration) that then mixes with what is taken up. Both est_va.m and N2_lung_coll have important changes and est_va.m resets VLinit0 and VTinit and we can just change that to the new volumes so this could be an easy swap. N2_lung_coll  has a VLprev which is given VL at the end of the function and we would need to give LUNG_VOL(i,1)=VL=tidal volume and LUNG_VOL(i,2)=tracheal volume and I think this can be as previously

**ChatGPT said:**

Yes, that is a sensible way to proceed. The existing `ALV_SURFACE_VALUE` can provide the initial, perfectly mixed lung state, while subsequent breaths exchange only a finite volume.

The mixing equation is:

```matlab
exchange_fraction = V_BREATH / V_GAS;

n_new = (1-exchange_fraction) .* n_old ...
      + exchange_fraction .* n_inspired;
```

So the old alveolar gas is not discarded. The fraction that remains mixes with the inspired gas taken in during the breath.

For example, if:

```matlab
V_BREATH = 0.25 * V_GAS;
```

then 75% of the previous lung gas remains and 25% is replaced by inspired gas.

## Important terminology

In `N2_lung_coll.m`:

```matlab
VL
```

is the current alveolar volume, not the tidal volume. It is returned by the lung-collapse model.

The stored values are:

```matlab
LUNG_VOL(i,1) = VL;   % current alveolar volume
LUNG_VOL(i,2) = VT;   % current tracheal/dead-space volume
```

The tidal volume should remain:

```matlab
V_BREATH
```

and should not replace `VL`.

Thus:

```
VLmax = maximum total respiratory volume
VTmax = maximum tracheal/dead-space volume
TAC   = maximum alveolar volume
VL    = current alveolar volume after compression
VT    = current tracheal volume after compression
V_BREATH = inspired volume during a breath
```

## What should happen during a breath?

A breath event could do the following:

```matlab
1. Use the current finite lung gas.
2. Replace/mix V_BREATH with inspired gas.
3. Re-establish the lung at the surface starting volume.
4. Reset the mechanical search state for the following dive.
```

The mechanical reset would use the existing variables:

```matlab
VLinit = VLinit0;
VLprev = VLprev0;
Pint   = Pint_init;
FLAGGA = 0;
```

and the surface volume would be:

```matlab
LUNG_VOL(i,1) = VLinit0;
LUNG_VOL(i,2) = VTinit;
```

Then, when the animal moves underwater, `N2_lung_coll` can again calculate the compressed values of `VL` and `VT`, and at the end it continues to update:

```matlab
VLprev = VL;
```

That part of `N2_lung_coll` can probably remain unchanged.

### The key modification to `est_va.m`

At present, `est_va.m` resets the lung whenever:

```matlab
P_AMB(i) == SURFACE_VALUE
```

That is no longer appropriate because the animal may be at the surface without breathing.

The reset should instead depend on the breath event:

```matlab
if breath_flag == 1
    % reset to the post-breath surface volume
else
    % retain/use the finite lung state
end
```

Therefore:

- surface plus breath → reset mechanical state;
- surface without breath → continue finite lung/gas iteration;
- underwater → use `N2_lung_coll` as before.

The same principle applies to `update_lung_gas.m`: `ALV_SURFACE_VALUE` should no longer be used simply because `Pamb == SURFACE_VALUE`. It should be used only during initialization, or possibly as the starting reference for a deliberate breath reset.

One correction to the earlier approach: we probably should not use the new helper as a separate replacement for all of the existing A–G logic immediately. The better sequence is:

1. Modify the surface reset in `est_va.m` so it responds to `breath_flag`, not merely surface pressure.
2. Modify `update_lung_gas.m` so the breath performs finite gas mixing.

3. Leave `N2_lung_coll.m` mostly unchanged initially.
4. Test surface periods without breaths.
5. Test a single breath.
6. Test repeated breath–dive cycles.

The next code change should therefore be to add `breath_flag` to the model's data stream and make it available to both `est_va.m` and `update_lung_gas.m`. We should not change `N2_lung_coll.m` until we see whether the existing volume state behaves correctly after the new breath reset.

Worked for 44s

---

**You said:**

I am not sure where VLinit0 us used nor VLprev0? I think they are initiated in Get_variables_ lung_coll but not sure they are used anywhere? I think by "forcing" all depths to be slightly deeper than 0m, surface, then we force the entire model to work. Then at a breath, we make P_AMB(i)=SURFACE_VALUE for an instant to take the breath, and then we go back down. This is not physiologicaly cheating as a marine mammal hardly every is at the surface and if they do, the lung will still be at something deeper than the surface and there is no breaths in between so long intervals. So having them "always" dive, come briefly to the surface to breathe is not only easier to edit for the model, it is a plausible physiological reality, so yes, breath_flag=1 equals P_AMB(i)=SURFACE_VALUE, or we can make them both be true for it to open the airways? What I am worried about is the surface without breath, and we need to make sure that this case has no old code that does a reset somewhere. if you still agree with your approach, then lets do the addition of breath_flag. I guess we still do not need a "dummy" data set?

**ChatGPT said:**

Yes, I agree with your revised approach. Treating the whale as being slightly submerged between breaths, with a brief modelled visit to the surface during a breath, is both simpler and physiologically defensible.

### `VLinit0` and `VLprev0`

You were correct that they are initialized in `GET_VARIABLES_LUNG_COLL_ZIP.m`:

```matlab
VLinit0 = V_LUNG - VTmax;
VLprev0 = VLprev;
```

Their direct uses are:

```matlab
est_va.m
```

uses `VLinit0` when the animal is at `SURFACE_VALUE`:

```matlab
LUNG_VOL(i,1) = VLinit0;
LUNG_VOL(i,2) = VTinit;
```

and:

```matlab
update_lung_gas.m
```

uses them when resetting after a surface period:

```matlab
VLinit = VLinit0;
VLprev = VLprev0;
Pint   = Pint_init;
FLAGGA = 0;
```

`VLprev0` is otherwise only a stored reference value. `VLprev` itself is updated in `N2_lung_coll.m`:

```matlab
VLprev = VL;
```

So these variables are important, but they are only used in a few specific places.

### Your proposed pressure strategy

The model can work like this:

```
Plain text

Normal time step:
    depth > 0
    finite lung reservoir
    use existing lung-collapse and gas-exchange calculations

Breath time step:
    breath_flag = 1
    temporarily set model pressure to SURFACE_VALUE
    exchange the specified breath volume
    reset the mechanical surface state

Next time step:
    return to shallow depth
    use the finite-reservoir diving calculation again
```

This removes the problematic “surface without breathing” situation by construction. A depth of 0.1–0.3 m is treated as underwater, even though lung compression is negligible.

I recommend that `breath_flag` be the actual trigger and that the model internally set the pressure to the surface value during that time step. In other words:

```
matlab

if breath_flag(i) == 1
    Pamb(2) = SURFACE_VALUE;
end
```

This is safer than requiring the input depth and `breath_flag` to independently agree exactly, because measured depth may not be exactly zero during a breath.

The model should then use an explicit condition such as:

```
matlab

if breath_flag(i) == 1
    % breath exchange and surface reset
else
    % existing finite-reservoir calculation
end
```

The old A/C surface-reset branch should eventually be removed or disabled. Otherwise, it will still replace the lung gas with `ALV_SURFACE_VALUE`, defeating the finite-breath calculation.

### Do we need a dummy dataset?

No. We do not need a separate dummy dataset.

For testing, we can add a temporary `breath_flag` vector in MATLAB:

```matlab
breath_flag = zeros(size(P_AMB));
```

and manually set one or two values to one:

```matlab
breath_flag(100) = 1;
breath_flag(250) = 1;
```

That lets us test the plumbing before actual breath-event data exist. Later, the same vector can come from a new column in the input data file.

The next step should therefore be to add `breath_flag` as an input to the time-step loop and pass it into `update_lung_gas` and `est_va`. We should first make the flag work with one artificial breath event, without yet changing the gas-mixing equations.

Worked for 40s

---

**You said:**

One questions, do we really need or care if the pressure is set to the surface_value as I think it is only used at that instance? The other option is to use both breath_flag and also set all depth to 0.1 meter except each breath. This allows us redundancy in breaths and in case we have a missed place where we have a surface_value at some point? We can "force" depth to be 0 at the breath. Not sure why you think this is safer? I am not sure what is "A/C" surface reset? I agree that we can then remove these values completely from the initialization file. OK so I will

create "breath_flag" inside `update_lung_gas` and `est_va` as global and then we later "pull" it from the data file later?

**ChatGPT said:**

Yes—your proposal is reasonable, and I agree that forcing the model depth to 0.1 m between breaths and 0 m during a breath is simpler.

I would use two safeguards:

```matlab
breath_flag(i) == 1
```

and:

```matlab
depth_model(i) = 0;
```

at the same time point.

For all other points:

```matlab
depth_model(i) = 0.1;
```

or, preferably, retain the measured depth if you still want the animal's real diving profile. The important point is that non-breathing points must not equal the model's exact `SURFACE_VALUE`.

We should not overwrite the original measured depth data. Instead, create a separate model-depth vector:

```matlab
depth_model = depth;
depth_model(:) = 0.1;
depth_model(breath_flag == 1) = 0;
```

Later, if you want the actual diving depths, we can use measured depth between breaths and still force only the breath points to zero.

### What I meant by the A/C reset

The “A/C surface reset” refers to these two branches in `update_lung_gas.m`:

```matlab
if Pamb(1)==SURFACE_VALUE && Pamb(2)==SURFACE_VALUE
    % case A
elseif Pamb(1)>SURFACE_VALUE && Pamb(2)==SURFACE_VALUE
    % case C
```

Both branches call:

```matlab
update_N2_case_A_or_C
update_O2_case_A_or_C
update_CO2_case_A_or_C
```

Those functions replace the existing lung gas with `ALV_SURFACE_VALUE`. That is the infinite-reservoir behaviour we want to eliminate.

With your proposed depth structure:

- normal points are at 0.1 m;
- breath points are at 0 m;
- there should be no ordinary surface-without-breath points;
- `breath_flag` explicitly identifies the actual breath.

Therefore, pressure can be used as a secondary check, but `breath_flag` should be the actual physiological trigger.

### How to create `breath_flag`

Do not create the values inside `update_lung_gas.m` or `est_va.m`. Those functions should read the flag, not invent it.

For the temporary test, initialize it once in the driver or initialization code:

```matlab
global BREATH_FLAG
```

```
BREATH_FLAG = zeros(N,1);
BREATH_FLAG(100) = 1;
BREATH_FLAG(250) = 1;
```

Then add the declaration in the functions that need it:

```matlab
global BREATH_FLAG
```

in both:

```
update_lung_gas.m
est_va.m
```

Later, when the data file has a breath column, replace the temporary initialization with:

```matlab
BREATH_FLAG = input_data(:,breath_column);
```

### About `ALV_SURFACE_VALUE`

Do not remove it completely yet. It is still useful for the initial condition at the first time step.

What we should remove later is its use during every surface time step through the A/C functions. The distinction is:

```
initial condition:
    ALV_SURFACE_VALUE may be used

later breath:
    finite mixing with inspired gas

surface without breath:
    should never reset the lung
```

So the next step is:

1. Add a temporary global `BREATH_FLAG`.
2. Create `depth_model`, with 0.1 m normally and 0 m at breath points.

3. Use `depth_model` to calculate `P_AMB`.
4. Do not yet delete the A/C functions.
5. Confirm that one artificial breath produces the expected surface-pressure transition.

After that, we can replace the A/C branch in `update_lung_gas.m` with the explicit breath branch.

Worked for 32s

---

**You said:**

I am not sure I agree with your adding a depth_model(:) variable, it is adding unnecessary variables to the whole modeling. If the animal is at 0.5 m and there is a breath, it either means the device is sitting in a place that is not close to the blow-hole or something is wrong. The pressure measures the animals actual position and if it measures close to the blow hole, it has to be close to 0 or 0 when there is a breath. Maybe the checking of the data first should verify that before running the model and then we can assure that there is not a breath when the animal is at 20 meter. But all breaths should be at a depth of less than a meter or so. So I think we can scrap the idea of the depth_model variable and just use breath_flag. Then we use the A/C reset for only one case and the others will all be with a "closed" lung. This means that if an animal takes a breath and then returns to the surface without breathing, the lung will remain closed and gases exchanged but no new air added, which is what we want. Then the "update_GAS_case_A_or_C" functions can be edited for the mixing and we need the past lung volume, and gas concentrations. So this past lung volume will depend on the duration of the past dive, for example, it could be 0.5 liter and then the animal takes a breath that is 10 liter and it total lung capacity then is 15 liters, the animal inhalaed 10 liters into a reservoir that could be 15 liters and the trick will be know if the 0.5 liters were exhaled first or the inhalation was done immediately before any exhale. We may have to test this a bit as we are getting into some assumptions about respiratory capacity in these animals, but let's go …

**ChatGPT said:**

I agree. We can drop `depth_model`. The measured depth should remain the model's pressure input, and `breath_flag` should identify when atmospheric exchange occurs.

The logic becomes:

```matlab
if BREATH_FLAG(i) == 1
    % breath: finite exchange with inspired air
else
    % no breath: closed finite lung reservoir
end
```

The existing A/C pressure tests should no longer control atmospheric exchange.

### Important hidden reset in `est_va.m`

We should also change the surface condition in `est_va.m`. It currently resets the lung whenever pressure equals `SURFACE_VALUE`:

```matlab
if P_AMB(i) == SURFACE_VALUE
```

That should become a breath condition:

```matlab
if BREATH_FLAG(i) == 1
    LUNG_VOL(i,1) = VLinit0;
    LUNG_VOL(i,2) = VTinit;
    va = VLinit0;
else
    va = N2_lung_coll(P_AMB(i)*101.3,i);
end
```

This ensures that a surface point without a breath does not reset the lung. If the depth data are correct, a true breath should also occur close to the surface, but `BREATH_FLAG` is the event that matters physiologically.

We can add a validation before the model starts:

matlab

```
breath_depths = depth(BREATH_FLAG == 1);

if any(breath_depths > 1)
    error('At least one breath occurs deeper than 1 m.');
end
```

The exact depth variable name may differ in your driver, but the principle is correct.

### One important consequence for the breath equation

Your example shows that the earlier simple fraction equation is not sufficient in all cases:

Plain text

```
old lung gas = 0.5 L
breath volume = 10 L
maximum capacity = 15 L
```

Here:

matlab

```
V_BREATH / V_old = 10 / 0.5 = 20
```

so we cannot simply replace 20 times the old lung volume. We need to define what happens mechanically during the breath.

There are two possible assumptions:

1. The animal exhales first, leaving a residual volume, then inhales 10 L.
2. The animal retains the old gas and adds 10 L, expanding the lung until the capacity is reached.

For the first prototype, I suggest:

Plain text

```
old lung gas is the residual gas
V_BREATH is added inspired gas
the final volume cannot exceed the permitted lung capacity
```

The gas-mole calculation would then be based on actual volumes:

matlab

```
n_old_residual   = concentration_old .* V_residual;
n_inspired       = inspired_fraction .* P_surface .* V_BREATH ./ RT;
n_after_breath   = n_old_residual + n_inspired;
```

The unresolved choice is the residual volume:

```matlab
V_residual = ?
```

It could be:

- the volume remaining after exhalation;
- the current compressed lung volume;
- a specified fraction of `VLmax`.

That is a genuine physiological assumption, so we should not hide it inside the code.

### What to do now

For now, add the flag plumbing but do not yet rewrite the gas equations:

1. Add a temporary global in the driver:

   ```matlab
   global BREATH_FLAG

   BREATH_FLAG = zeros(N,1);
   ```

2. Add one artificial test breath:

   ```matlab
   BREATH_FLAG(100) = 1;
   ```

3. Add:

   ```matlab
   global BREATH_FLAG
   ```

   to both `update_lung_gas.m` and `est_va.m`.

4. Change the surface reset in `est_va.m` to depend on `BREATH_FLAG`, not only on `P_AMB`.

5. In `update_lung_gas.m`, use the breath flag to select the future modified A/C functions; for now, leave their existing gas calculation unchanged.
6. Check that a non-breathing point at shallow depth uses the closed-lung iteration and does not call the old surface reset.

You do not need a dummy dataset. The temporary flag vector with one artificial breath is enough to test the structure. Later, we can replace it with a data-file column.

Worked for 57s

---

**You said:**

About the lung volume, let me define that a bit more. Respiration in marine mammals is, most if not always, starting with an exhale followed by an inhalation. If the lung volume in an animal is 0.5 liter as the animal has come to the surface, this must have been a VERY long dive or from a previously very low diving lung volume. What the animal then does is inhale (likely no more exhale as there is about 5-10% of the total lung volume that cannot be exhaled), so the 0.5 liter is added to 10 liter of inhalaed air and mixes. The diving lung volume now is 10.5 liters of mixed air and the tracheal volume VT will always be the same at the surface and only changes volume during diving when it is compressed. So we need to "fix" these values during the BREATH_FLAG(i)==1 call. For your suggested way forward, I am nto sure where I should add the "BREATH_FLAG" and in which call functions or initialize in the general function that creates all variables? Also, then the rest of the procedure requires data or the model will not run and then we likely should create a dummy data set?

Worked for 1m 22s >

---

**ChatGPT said:**

That clarification makes the volume treatment much clearer.

At a breath, we can treat the current lung gas as the residual gas after exhalation. Then the animal inhales `V_BREATH`, and the two gas volumes mix:

Plain text

```
old residual gas + inspired breath = new lung gas
```

For your example:

Plain text

```
old residual volume = 0.5 L
inspired volume     = 10.0 L
new mixed volume    = 10.5 L
```

The tracheal volume at the surface is reset to `VTinit`. Therefore, at a breath:

matlab

```
V_old_total = LUNG_VOL(i-1,1) + LUNG_VOL(i-1,2);
V_new_total = V_old_total + V_BREATH;

LUNG_VOL(i,2) = VTinit;
LUNG_VOL(i,1) = V_new_total - VTinit;
```

Here `LUNG_VOL(i,1)` remains the alveolar volume and `LUNG_VOL(i,2)` remains the tracheal volume. We should not call `VL` the tidal volume.

We will also reset the mechanical state for the new breath:

matlab

```
VLinit = LUNG_VOL(i,1);
VLprev = VLinit - 0.5;
Pint   = Pint_init;
FLAGGA = 0;
```

`VLinit0` should remain the original reference value for now. It should no longer automatically replace the current volume during every surface time step. After the new system works, `VLinit0` and `VLprev0` may become unnecessary and can be removed carefully.

### Where to create `BREATH_FLAG`

Do not create it inside `update_lung_gas.m` or `est_va.m`. Those functions should read it.

The best temporary location is `beakedwhale_tissue_kinetics1.m`, immediately after:

```matlab
N = length(s);
```

Add:

```matlab
global BREATH_FLAG

BREATH_FLAG = zeros(N,1);
```

For the first test, manually add one breath after checking that the selected row is shallow:

```matlab
BREATH_FLAG(100) = 1;
```

Replace `100` with a real row where:

```matlab
s(100,2) < 1
```

Then add this validation:

```matlab
depth = s(:,2);
depth(depth < 0) = 0;
s(:,2) = depth;

if any(depth(BREATH_FLAG == 1) > 1)
    error('A breath event occurs deeper than 1 m.');
end
```

Later, when the data file has a breath column, this temporary line:

```matlab
BREATH_FLAG = zeros(N,1);
```

will become something like:

```matlab
BREATH_FLAG = s(:,6);
```

depending on where the new column is placed.

### Add the global declaration to the functions

At the beginning of each function that needs the flag, add:

```matlab
global BREATH_FLAG
```

This applies to:

```
GET_GAS_FLUX_1.m
update_lung_gas.m
est_va.m
```

`GET_VARIABLES_ZIP_NEW.m` is not the best place to initialize the vector because it defines animal-specific constants, whereas `BREATH_FLAG` is a time-series input with length `N`.

### How `est_va.m` should use it

The current surface test:

```matlab
if P_AMB(i) == SURFACE_VALUE
```

should eventually become:

```matlab
if BREATH_FLAG(i) == 1
    % Set surface tracheal volume and post-breath alveolar volume
else
    % Use the closed-lung mechanics
    va = N2_lung_coll(P_AMB(i)*101.3,i);
end
```

This ensures that a shallow point without a breath does not reset the lung.

### Do we need a dummy dataset?

No. We can use the existing whale-12 dataset and create only a temporary flag vector:

```matlab
BREATH_FLAG = zeros(N,1);
BREATH_FLAG(test_row) = 1;
```

That is not a dummy dataset; it is simply a temporary test event. Once the model successfully handles one artificial breath, we can add a genuine `breath_flag` column to the data file.

The next coding step should be the flag initialization and validation only. Then we can modify `est_va.m` so the breath event creates the new mixed lung volume. After that, we will modify the A/C gas functions to use the previous lung gas and the newly inhaled volume.

**You said:**

one thing, old and new volume cannot become larger than the total lung capacity, so if old volume is 9 liter (alveolar + tracehal volume), and the animal inahales 10 liters, we need to clearly define residcual volume and maybe that should be the tracheal volume and then 9liters - tracehal volume is exhaled and 10 liters + tracehal volume in the lung after. We need to amke sure that we get that algebra and conditions correct, but maybe we start simple and always to inhale to total lung capacut, and then breath fraction =1. ok now I will do what you provided

**ChatGPT said:**

Exactly. The volume must be capped at `VLmax`, and we need to distinguish between residual volume and inspired volume.

For the simple first version, I suggest:

- At the start of a breath, retain only the tracheal volume.
- Exhale the previous alveolar gas.
- Inhale until total lung volume reaches `VLmax`.

- Keep `VTinit` as the surface tracheal volume.

The algebra is then:

```matlab
V_residual = VTinit;
V_final    = VLmax;
V_breath   = V_final - V_residual;
```

Therefore:

```matlab
V_breath = VLmax - VTinit;
```

For example, if:

```
old total lung volume = 9 L
VTinit                = 0.5 L
VLmax                 = 15 L
```

then:

```
old alveolar volume exhaled = 9 - 0.5 = 8.5 L
residual gas retained       = 0.5 L
inspired volume added       = 15 - 0.5 = 14.5 L
final total volume          = 15 L
```

The old alveolar gas is therefore removed, while the residual tracheal gas mixes with the inspired gas.

For this initial model, we can define:

```matlab
BREATH_FRACTION = 1.0;
V_BREATH = BREATH_FRACTION * (VLmax - VTinit);
```

This means that `BREATH_FRACTION = 1` represents a full inhalation to total lung capacity. A later fractional breath could use, for example:

```matlab
BREATH_FRACTION = 0.7;
```

with:

```matlab
V_BREATH = 0.7 * (VLmax - VTinit);
```

The final volume would then be:

```matlab
V_final = VTinit + V_BREATH;
```

and it cannot exceed `VLmax`.

This is slightly different from defining `BREATH_FRACTION = V_BREATH/VLmax`; for the first implementation, I recommend defining it relative to the inhalable volume above residual tracheal volume. We can document that clearly so the meaning does not become ambiguous.

For now, proceed with the `BREATH_FLAG` initialization and validation. Do not yet modify the volume equations or the old A/C functions. Once the flag is working, we will implement the breath-volume calculation with these limits:

```matlab
V_final = VTinit + V_BREATH;
V_final = min(V_final, VLmax);
```

That will prevent the lung from ever exceeding total capacity.

Worked for 28s

---

**You said:**

1 selection

but this means I would have to run the model? it is a half hour run so why not fix a dummy data set? or if not, then what do you mean by checking that it is close to the surface?

**ChatGPT said:**

You do not need to run the full model just to check whether row 100 is near the surface.

`BREATH_FLAG(100) = 1` only marks row 100 as a temporary test breath. Before running the model, inspect the depth directly in MATLAB.

After loading the whale-12 data, run:

matlab

```
s = GET_DIVE_DATA_LOAD_TISSUE_KIN(12);

s(100,2)
```

The result is the depth at row 100. If it is less than 1 m, that row is suitable for the temporary test.

You can find a suitable row automatically:

matlab

```
candidate_rows = find(s(:,2) < 1);

candidate_rows = candidate_rows(candidate_rows > 1);

test_row = candidate_rows(1)
test_depth = s(test_row,2)
```

Then use:

matlab

```
BREATH_FLAG = zeros(size(s,1),1);
BREATH_FLAG(test_row) = 1;
```

Also add a safety check:

matlab

```
if any(s(BREATH_FLAG == 1,2) > 1)
    error('A breath event occurs deeper than 1 m.');
end
```

This check happens before the expensive model run.

I would not create a shortened dummy dataset yet. The model depends on the dive-summary row numbers, so shortening the data would also require modifying the summary file and could create unrelated problems. A temporary `BREATH_FLAG` vector using the real data is simpler and safer.

However, this only tests that the flag is correctly created and located at a shallow point. To test the new breath branch itself, we will eventually need one complete model run—or later we can create a small isolated test function for the lung-volume and gas-mixing calculations without running the entire 30-minute model.

Worked for 20s

---

**You said:**

rows 90-93 are at the surface, after that the animals dives.

**ChatGPT said:**

Good. Use one of those rows for the temporary test—for example:

matlab

```
BREATH_FLAG = zeros(size(s,1),1);
BREATH_FLAG(90) = 1;
```

Rows 90–93 are all shallow, so row 90 is valid.

This is useful because it tests:

Plain text

```
row 89 → row 90: breath event
row 90 → row 91: surface, but no new breath
row 93 → row 94: descent
```

That lets us check both the new breath treatment and whether a surface point without `BREATH_FLAG = 1` accidentally triggers the old reset.

Add the validation:

```matlab
if any(s(BREATH_FLAG == 1,2) > 1)
    error('A breath event occurs deeper than 1 m.');
end
```

For now, mark only one row. Do not mark rows 90–93 as breaths, because those rows represent time at the surface, not necessarily four separate breaths. Later, the actual `breath_flag` data will identify the individual breath events.

Worked for 17s

---

**You said:**

I am confused, so I need to run the model...but how do i avoid that it does not run the entire series? And where do I add the code above?

**ChatGPT said:**

You do not need to run the full model yet. First, only check the data row.

In MATLAB, with the `src` folder as the current folder, run:

```matlab
s = GET_DIVE_DATA_LOAD_TISSUE_KIN(12);

s(90,2)
```

This loads the whale-12 data but does not run the physiological model. The output should be the depth at row 90.

Now add the temporary flag in:

```Plain text
beakedwhale_tissue_kinetics1.m
```

Near the top, add `BREATH_FLAG` to the global declarations:

matlab

```
global BREATH_FLAG
```

Then find:

matlab

```
N = length(s);
```

Immediately below it, add:

matlab

```
BREATH_FLAG = zeros(N,1);
BREATH_FLAG(90) = 1;

if any(s(BREATH_FLAG == 1,2) > 1)
    error('A breath event occurs deeper than 1 m.');
end
```

So it should look like:

matlab

```
N = length(s);

BREATH_FLAG = zeros(N,1);
BREATH_FLAG(90) = 1;

if any(s(BREATH_FLAG == 1,2) > 1)
    error('A breath event occurs deeper than 1 m.');
end
```

This creates one temporary breath event at row 90. It does not yet change the model because we have not connected `BREATH_FLAG` to the gas-update functions.

Therefore:

- The command-window test checks the depth without running the model.
- The code in `beakedwhale_tissue_kinetics1.m` prepares the flag.
- Do not run the 30-minute model yet.

- The next step is to connect the flag to `est_va.m` and `update_lung_gas.m`, then we will run the full whale-12 model to test the actual breath behavior.

Worked for 21s

---

**You said:**

So i do not need to run this yet? and then we do the next step?

**ChatGPT said:**

Correct. You do not need to run the full model yet.

First, save the changes in `beakedwhale_tissue_kinetics1.m`. Then, in MATLAB, run only:

```matlab
s = GET_DIVE_DATA_LOAD_TISSUE_KIN(12);
s(90,2)
```

This confirms that row 90 is shallow without running the model.

Once that works, we can connect `BREATH_FLAG` to `est_va.m` and `update_lung_gas.m`. Only after that connection will we run the full model.

**You said:**

ok lets add that

**ChatGPT said:**

Yes. Now we can connect the temporary flag to the two functions, but we will initially keep the old gas-reset calculation only as a temporary test. We are testing the structure first; the actual finite-breath mixing comes next.

### 1. Edit `est_va.m`

Near the top, add:

```matlab
global BREATH_FLAG
```

Find:

```matlab
if P_AMB(i)==SURFACE_VALUE
```

Change it to:

```matlab
if BREATH_FLAG(i)==1
```

Leave the existing contents of that branch unchanged for now:

```matlab
if BREATH_FLAG(i)==1
    LUNG_VOL(i,1)=VLinit0;
    LUNG_VOL(i,2)=VTinit;
    va=VLinit0;
else
    va=N2_lung_coll(P_AMB(i)*101.3,i);
end
```

This means only the temporary flagged breath at row 90 causes the surface-volume reset. A later surface point without a breath will use `N2_lung_coll`.

### 2. Edit `update_lung_gas.m`

Near the top, add:

```matlab
global BREATH_FLAG
```

Then replace the first condition:

```matlab
if Pamb(1)==SURFACE_VALUE && Pamb(2)==SURFACE_VALUE
```

with:

```matlab
```

```
if BREATH_FLAG(i)==1
```

Delete the following `elseif` branch for case C, including its duplicate reset code.

The structure should become:

```matlab
if BREATH_FLAG(i)==1

    % Keep the existing A/C gas-update calls temporarily
    [P_N2_lung_t,n_N2_lung_t,P_N2_art_t,n_N2_art_t] = ...
        update_N2_case_A_or_C(B_VOL_PACKAGE_VEN,SN2_BLOOD,rt);

    [P_O2_lung_t,n_O2_lung_t,P_O2_art_t,n_O2_art_t] = ...
        update_O2_case_A_or_C(B_VOL_PACKAGE_VEN,SO2_BLOOD,rt);

    [P_CO2_lung_t,n_CO2_lung_t,P_CO2_art_t,n_CO2_art_t] = ...
        update_CO2_case_A_or_C(B_VOL_PACKAGE_VEN,SCO2_BLOOD,rt);

    VLinit=VLinit0;
    VLprev=VLprev0;
    Pint=Pint_init;
    FLAGGA=0;

else
```

This is only a structural test. The A/C functions still use `ALV_SURFACE_VALUE`, so this is not yet the final finite-breath physiology.

### 3. Run the model

Now the flag is actually used. Run the whale-12 model normally.

The test should confirm that:

- row 90 uses the breath branch;
- row 91, despite still being at the surface, does not use the old A/C branch;
- rows after the surface interval use the finite iteration;
- the model completes without errors.

If the model fails at row 91 or immediately after the breath, that will show us how `N2_lung_coll` behaves when called at surface pressure without the old reset. That is useful information before we implement the new volume and gas-mixing equations.

Worked for 34s

**You said:**

this will crash the model as you will asphyxiate the whale. You have one breath in several hours of diving....so this will nto work well and maybe we can "force quit" the model by forcing the for loop in Get_gas_flux end after 10000 rows?

**ChatGPT said:**

You are right. With only one artificial breath followed by hours of diving, the model would eventually deplete the lung gases and the test would be physiologically meaningless.

For this structural test, limit the time-step loop.

In `GET_GAS_FLUX_1.m`, find:

```matlab
for i=2:N
```

Temporarily replace it with:

```matlab
MAX_TEST_STEPS = 100;

for i=2:min(N,MAX_TEST_STEPS)
```

Because the test breath is at row 90, 100 steps is enough to test:

- the breath at row 90;
- the following surface points;
- the beginning of the descent.

If you want to follow the dive slightly further, use:

```matlab
MAX_TEST_STEPS = 1000;
```

There is no need to use 10,000 yet.

Keep:

matlab

```
BREATH_FLAG(90) = 1;
```

The model will then stop after the selected number of rows rather than running through the entire record.

This run is only a structural test. Do not interpret the saved physiological output because the simulation is intentionally truncated. After testing, restore the original loop:

matlab

```
for i=2:N
```

Also add a visible message so you remember this is a test:

matlab

```
disp(['TEST RUN: model stopped at row ', num2str(min(N,MAX_TEST_STEPS))])
```

Place that after the loop, before the function finishes.

So the temporary sequence is:

matlab

```
MAX_TEST_STEPS = 100;

for i=2:min(N,MAX_TEST_STEPS)
    % existing model loop
end

disp(['TEST RUN: model stopped at row ', ...
      num2str(min(N,MAX_TEST_STEPS))])
```

This lets us test the new breath flag without subjecting the simulated whale to several hours of diving after a single breath.

Worked for 29s

---

**You said:**

ok, but then how do we know the model actually did what we wanted unless we compare values from before and

after? Maybe we should save some of the data so we can make comparisons and this would also help future debugging?

**ChatGPT said:**

Yes—this is a very good idea. We should save a small diagnostic table around the artificial breath so we can verify exactly what changed.

Add the following code in `beakedwhale_tissue_kinetics1.m` immediately after:

matlab

```
GET_GAS_FLUX_1(dsum);
```

matlab

```
% Diagnostic output around the temporary breath event
debug_rows = 88:95;
debug_rows = debug_rows(debug_rows <= N);

BREATH_DEBUG = [ ...
    debug_rows(:), ...
    BREATH_FLAG(debug_rows), ...
    P_AMB(debug_rows), ...
    LUNG_VOL(debug_rows,1), ...
    LUNG_VOL(debug_rows,2), ...
    LUNG_VOL(debug_rows,3), ...
    N2_ARRAY(debug_rows,5,1), ...
    N2_ARRAY(debug_rows,5,2), ...
    O2_ARRAY(debug_rows,5,1), ...
    O2_ARRAY(debug_rows,5,2), ...
    CO2_ARRAY(debug_rows,5,1), ...
    CO2_ARRAY(debug_rows,5,2)];
```

The columns will be:

Plain text

```
1   row number
2   BREATH_FLAG
3   ambient pressure
4   alveolar volume
5   tracheal volume
6   internal pressure
7   lung PN2
```

```
8   lung N2 moles
9   lung PO2
10  lung O2 moles
11  lung PCO2
12  lung CO2 moles
```

After the short test run, inspect it with:

matlab

```
load('breath_debug_test.mat')
BREATH_DEBUG
```

The most important rows are:

Plain text

```
89 = immediately before the breath
90 = temporary breath event
91 = surface without a breath
94 = beginning of descent
```

At this temporary stage, we are checking the model structure rather than the final gas-mixing physiology. We should confirm that:

- row 90 has `BREATH_FLAG = 1`;
- row 90 enters the breath branch;
- row 91 does not automatically use the old A/C branch;
- the model reaches the descent;
- the lung-volume and gas arrays remain finite.

Later, after implementing the proper finite breath, we will use the same diagnostic output to check that:

Plain text

```
post-breath total volume ≤ VLmax
tracheal volume = VTinit at the surface
alveolar volume = total volume − VTinit
old residual gas + inspired gas = new lung gas
```

This diagnostic file will also be useful for future debugging. For now, use the short run with:

matlab

```
MAX_TEST_STEPS = 100;
```

and restore the full loop only after the routing test succeeds.

Worked for 37s

---

**You said:**

one issue here and the interal pressure is now 0 at the breath and it should be ambient pressure so this may be someting to watch for and i will leavde the debug for now at 1000 steps and then we continue tomorrow

**ChatGPT said:**

Good catch. The zero internal pressure is probably because the breath branch in `est_va.m` bypasses `N2_lung_coll.m`, which normally writes:

matlab

```
LUNG_VOL(i,3) = Pint;
LUNG_VOL(i,4) = PAMB;
```

The breath branch currently sets only the alveolar and tracheal volumes, so `LUNG_VOL(i,3)` remains zero.

For tomorrow, we should set the pressure explicitly during the breath:

matlab

```
Pint = P_AMB(i) * 101.3;
LUNG_VOL(i,3) = Pint;
LUNG_VOL(i,4) = P_AMB(i) * 101.3;
```

The conversion is needed because `P_AMB` is stored in atmospheres, whereas `Pint` and `LUNG_VOL(:,3:4)` use kPa.

At the surface this should correspond approximately to:

matlab

```
Pint = Pint_init;
```

We should check the units carefully tomorrow, but the zero value is a programming issue—not a physiological result.

Leaving the diagnostic run at 1,000 steps is fine. The saved table should help us inspect the pressure and volume transition before we continue.

Worked for 18s

---

**You said:**

ok, hasta manana

**ChatGPT said:**

¡Hasta mañana! We'll continue by fixing the breath-step pressure and then check the diagnostic output.